

The block below imports a necessary module and checks if you are in the right directory. If you do not see a list of python files when you run the block, please make sure you have uploaded the zip file and unzipped it.

```
In [1]: import matplotlib.pyplot as plt
```

```
%pwd  
%ls
```

```
atomgenerator.py  embedmodel.py      kbencoder.py       nnunifier.py  
autoencoder.py   evaluate.py         kbparser.py        reasoner.py  
basictypes.py    gameofthrones.txt  knowledgebase.py   termwalk.py  
chainbased.py    gen_queries.py     mr_back_reasoner.py vocab.py  
'CSE 327 P4.ipynb' helpers/           nnreasoner.py
```

If you want to create a random knowledge base, run the following code.

Creates the files:

- randomKB.txt - A random KB generated using 10 predicates and 100 constants.
- vocab.pkl - Indexes the full set of predicates, constants and variables used by the knowledge base.

```
In [2]: !python kbencoder.py --generate_kb --new_vocab --save_vocab  
!head randomKB.txt
```

```
Classes: 1000000000.0  
Max arity: 4  
KB generated (150)  
p3(a40, a79).  
p7(a93, a47).  
p4(a12).  
p4(a10).  
p3(a62, a6).  
p9(a32, a0).  
p5(a5).  
p8(a99, a69, a27, a21).  
p2(a13, a30).  
p0(a51, a54, a21).
```

The following code prepares an existing knowledge base (KB) for use. To use a KB of your choosing, replace gameofthrones.txt with the file name of any KB you want to use.

Creates the file:

- vocab.pkl - Indexes the full set of predicates, constants and variables used by the knowledge base.

```
In [3]: !python kbencoder.py --kb_path gameofthrones.txt --vocab_from_kb --save_vocab
```

Creating vocabulary from last gameofthrones.txt knowledge base

Train a neural net to convert first-order logic atoms into meaningful numeric vectors. We call these vectors "embeddings," and the learned model the embedding model. The code creates random atoms and then uses a mechanism known as triplet loss to place unifying atoms closer together.

Creates the files:

- rKB_model.pth - Saves the learned weights for the embedding model
- training_loss-45-106-2-50.png - A plot of the training loss after each epoch. This file's name depends on the vocabulary of your KB: the four numbers are number of predicates, number of constants, max arity of the predicates, and the embedding size used. If you changed the KB, you'll need to look up the name in your drive and update the code block that displays this file.

```
In [4]: !python kbencoder.py --kb_path gameofthrones.txt --train_unification_model
```

```
Using cuda device
```

```
Uni model embed size: 50
```

```
[████████████████████] 3500/3500 Anchors generated
```

```
[████████████████████] 70000/70000 Triplets generated
```

```
[████████-----] 18104/70000 Triplet encodings
```

```
IOPub data rate exceeded.
```

```
The Jupyter server will temporarily stop sending output  
to the client in order to avoid crashing it.
```

```
To change this limit, set the config variable
```

```
`--ServerApp.iopub_data_rate_limit`.
```

```
Current values:
```

```
ServerApp.iopub_data_rate_limit=1000000.0 (bytes/sec)
```

```
ServerApp.rate_limit_window=3.0 (secs)
```

```
[████████████████████-----] 48277/70000 Triplet encodings
```

```
IOPub data rate exceeded.
```

```
The Jupyter server will temporarily stop sending output  
to the client in order to avoid crashing it.
```

```
To change this limit, set the config variable
```

```
`--ServerApp.iopub_data_rate_limit`.
```

```
Current values:
```

```
ServerApp.iopub_data_rate_limit=1000000.0 (bytes/sec)
```

```
ServerApp.rate_limit_window=3.0 (secs)
```

```
[████████████████████] 70000/70000 Triplet encodings
```

```
Early stopping triggered-----] 33.1% Epoch | [Epoch 166/500, Training Loss:  
0.0629, Validation Loss: 0.2583]
```

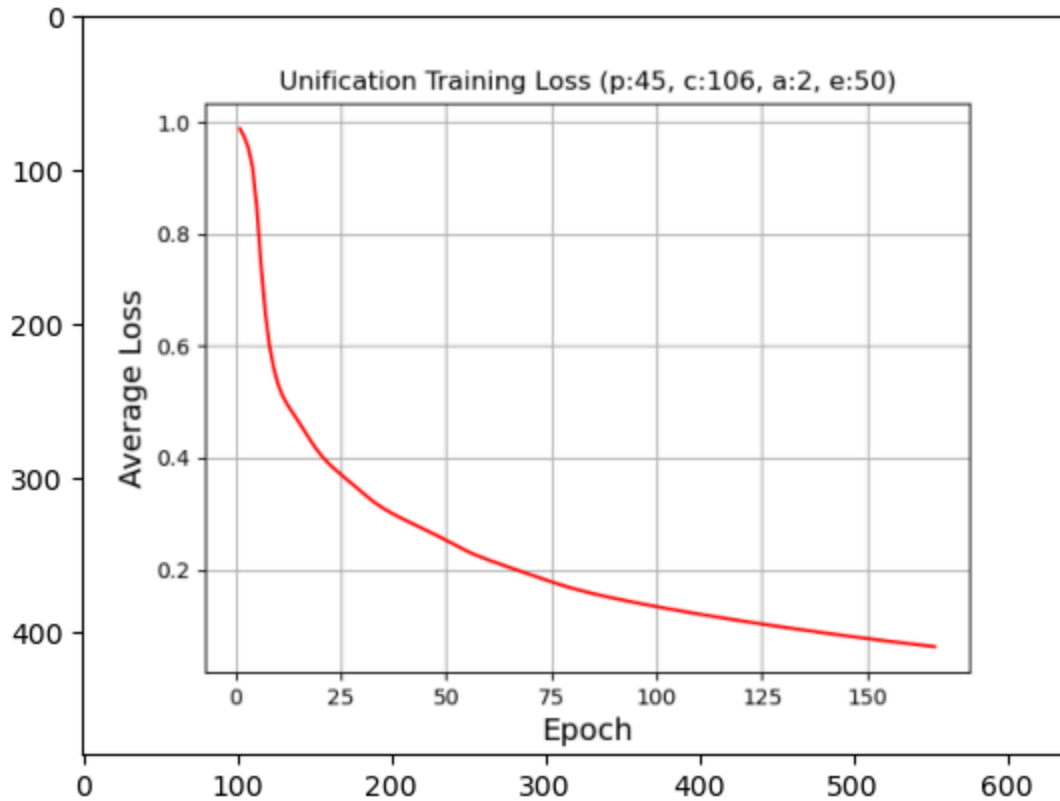
```
Saved model.
```

```
Saved plot.
```

```
unification model generated
```

```
In [8]: img = plt.imread('training_loss-45-106-2-50.png')  
plt.imshow(img)
```

Out[8]: <matplotlib.image.AxesImage at 0x7f7860ff1450>



Generate a set of train and test queries from your knowledge base. This will also generate the deductive closure (all facts that can be inferred) of your KB using forward-chaining.

Produces the files:

- all_facts.txt - A list of all facts that can be inferred from the KB. This is used to generate queries that are guaranteed to have an answer.
- train_queries.txt - A set of training queries where each will have at least one answer.
- test_queries.txt - A set of test queries where each will have at least one answer.

In [9]: `!python kbencoder.py --kb_path gameofthrones.txt -g`

```
Generating facts list...
Forward chaining depth: 1
New facts: 442
Forward chaining depth: 2
New facts: 175
Forward chaining depth: 3
New facts: 82
Forward chaining depth: 4
New facts: 12
Forward chaining depth: 5
New facts: 1
Forward chaining depth: 6
New facts: 2
Forward chaining depth: 7

Facts list generated
714 total facts. Max depth = 6

Number of queries: 200
100 training queries generated
100 testing queries generated
```

Create a training set for the scoring model. The examples will consist of goal, rule, score pairs, where a score of 1 means the rule can solve the goal, and 0 means it cannot. It runs the full set of training queries with a randomized backward-chaining process. This step can sometimes take a long time and is not able to take advantage of a GPU (with the gameofthrones KB it took ~45 minutes).

Creates a file:

- `mr_train_examples.csv` - Contains a line for each goal/rule/score example.

```
In [1]: !python kbencoder.py --kb_path gameofthrones.txt -p
```

running training queries...

Negative facts will be generated

1: [child(_EX2, rhaegar_targaryen)]

min depth: 2

Nodes: 5

Answers: 3

2: [father(_EX6, aegon_targaryen)]

min depth: 3

Nodes: 5

Answers: 1

3: [descendant(robert_baratheon, X), parent(X, Y), father(steffon_baratheon, Y), mother(X, Y)]

min depth: 3

Nodes: 500

Answers: 2

4: [who_killed_daenerys(_EX4)]

min depth: 2

Nodes: 3

Answers: 1

5: [child(loras_tyrell, _EX4)]

min depth: 2

Nodes: 4

Answers: 2

6: [parent(X, Y), mother(X, sansa_stark), descendant(sansa_stark, X), parent(eddard_stark, Y), daughter(Y, X)]

min depth: 1

Nodes: 105

Answers: 104

7: [father(Z, _EX1)]

min depth: 3

Nodes: 160

Answers: 54

8: [son(X, Y), descendant(tyrion_lannister, X), descendant(joffery_lannister, X), descendant(joffery_lannister, Y)]

min depth: 3

Nodes: 178

Answers: 72

9: [ancestor(_EX4, _EX5)]

min depth: 2

Nodes: 23487

Answers: 230

10: [son(Y, _EX3)]

min depth: 3

Nodes: 121

Answers: 72

11: [ancestor(X, Y), ancestor(X, Z), son(Y, X), love(jorah_mormont, Z)]

min depth: 2
Nodes: 100171
Answers: 128

12: [son(X, aegon_v_targaryen), ancestor(X, Y), parent(rhaella_targaryen, Y)]
min depth: 3
Nodes: 52
Answers: 3

13: [ancestor(unknown_mother_greyjoy, X), son(X, Y), descendant(rodrick_greyjoy, Y)]
min depth: 2
Nodes: 21562
Answers: 5

14: [descendant(_EX0, _EX5)]
min depth: 3
Nodes: 41293
Answers: 225

15: [child(obara_sand, X), father(lewyn_martell, X)]
min depth: 2
Nodes: 4
Answers: 2

16: [descendant(rhaenys_targaryen, Y)]
min depth: 3
Nodes: 15253
Answers: 26

17: [ancestor(Y, X), ancestor(unknown_father_greyjoy, X), ancestor(Y, theon_greyjoy), son(theon_greyjoy, Y)]
min depth: 4
Nodes: 23649
Answers: 210

18: [ancestor(X, _EX2)]
min depth: 4
Nodes: 25268
Answers: 214

19: [ancestor(unknown_mother_lannister, _EX3)]
Restart...
min depth: 6
min depth: 4
Nodes: 44592
Answers: 11

20: [descendant(_EX6, _EX3)]
min depth: 3
Nodes: 26609
Answers: 184

21: [ancestor(unknown_mother_martell, _EX5)]
Restart...
min depth: 4
Nodes: 57942

Answers: 20

22: [descendant(tommen_lannister, X), ancestor(X, joffery_lannister)]

min depth: 3

Nodes: 77

Answers: 16

23: [descendant(_EX5, X)]

min depth: 5

Nodes: 47650

Answers: 219

24: [descendant(X, Y), descendant(X, olenna_tyrell), child(X, Y), ancestor(Y, X), ancestor(Y, loras_tyrell)]

min depth: 5

Nodes: 11413

Answers: 196

25: [descendant(Y, X), ancestor(X, yara_greyjoy), parent(Z, Y), descendant(_EX0, Z), daughter(yara_greyjoy, _EX0)]

min depth: 5

Nodes: 10478

Answers: 182

26: [ancestor(Z, X)]

min depth: 4

Nodes: 1575

Answers: 190

27: [child(_EX6, oberyntmartell)]

min depth: 2

Nodes: 7

Answers: 5

28: [father(_EX5, _EX6)]

min depth: 3

Nodes: 103

Answers: 54

29: [ancestor(_EX5, _EX1)]

min depth: 2

Nodes: 39759

Answers: 238

30: [status(night_king, X), status(Y, X), son(Y, aegon_v_targaryen)]

min depth: 1

Nodes: 2

Answers: 1

31: [descendant(X, Y), parent(steffon_baratheon, X), child(shireen_baratheon, X), ancestor(Y, shireen_baratheon)]

Restart...

min depth: 3

Nodes: 18322

Answers: 210

32: [female(X)]

min depth: 1

Nodes: 33

Answers: 32

33: [status(Y, X), status(daeron_targaryen, X)]

min depth: 1

Nodes: 80

Answers: 79

34: [child(X, rickard_stark), descendant(X, rickard_stark), parent(X, Y), ancestor(unknown_mother_stark, Y), parent(unknown_mother_stark, X)]

min depth: 2

Nodes: 6

Answers: 4

35: [child(tyrion_lannister, tywin_lannister)]

min depth: 2

Nodes: 3

Answers: 1

36: [descendant(X, unknown_mother_targaryen), descendant(Y, X), ancestor(X, viserys_targaryen), ancestor(Y, rhaenys_targaryen), descendant(aegon_targaryen, Y)]

min depth: 3

Nodes: 20548

Answers: 14

37: [ancestor(Y, X), status(X, dead), son(X, Y), ancestor(olenna_tyrell, X)]

Restart...

min depth: 2

Nodes: 100195

Answers: 106

38: [ancestor(X, rickon_stark), son(Y, X), descendant(Y, X), parent(unknown_mother_stark, Y)]

Restart...

min depth: 4

Nodes: 3952

Answers: 4

39: [ancestor(X, Y), parent(X, Y), father(X, Y), parent(cassana_estermont, X)]

min depth: 2

Nodes: 43442

Answers: 198

40: [ancestor(_EX3, X)]

Restart...

min depth: 2

Nodes: 1449

Answers: 188

41: [ancestor(X, daeron_targaryen), ancestor(X, viserys_targaryen), descendant(viserys_targaryen, X)]

min depth: 2

Nodes: 100130

Answers: 2

42: [status(thoros_of_myr, X), status(aerys_targaryen, X), status(nymeria_sand, X), status(Y, X), ancestor(joanna_lannister, Y)]
min depth: 1
Nodes: 2
Answers: 1

43: [father(Z, _EX1)]
min depth: 3
Nodes: 160
Answers: 54

44: [father(X, nymeria_sand), child(obara_sand, X)]
min depth: 3
Nodes: 5
Answers: 1

45: [kill(X, Y), kill(arya_stark, X), status(Y, alive), creatures(Y)]
min depth: 1
Nodes: 8
Answers: 7

46: [who_betrayed_daenerys(_EX0)]
min depth: 2
Nodes: 7
Answers: 5

47: [son(_EX4, Z)]
min depth: 3
Nodes: 121
Answers: 72

48: [son(daeron_targaryen, _EX2)]
min depth: 3
Nodes: 5
Answers: 2

49: [father(X, _EX5)]
min depth: 3
Nodes: 160
Answers: 54

50: [descendant(_EX1, kevan_lannister)]
Restart...
min depth: 3
Nodes: 13
Answers: 3

51: [child(sarella_sand, _EX4)]
min depth: 2
Nodes: 4
Answers: 2

52: [love(Y, X), who_loved_daenerys_and_was_corresponded(X), daughter(Y, aerys_targaryen), love(X, Y), who_loved_daenerys(X)]
min depth: 1

Nodes: 12
Answers: 11

53: [ancestor(Y, joffery_lannister)]
Restart...
min depth: 4
Nodes: 26027
Answers: 28

54: [child(_EX4, obery_n_martell)]
min depth: 2
Nodes: 7
Answers: 5

55: [descendant(aerys_targaryen, X), father(X, duncan_targaryen)]
min depth: 3
Nodes: 4448
Answers: 2

56: [son(X, doran_martell), ancestor(Y, X), son(X, Y), ancestor(unknown_mother_martell, X)]
min depth: 3
Nodes: 50
Answers: 1

57: [child(_EX5, _EX2)]
min depth: 2
Nodes: 106
Answers: 104

58: [daughter(cersei_lannister, _EX1)]
min depth: 3
Nodes: 6
Answers: 2

59: [mother(Y, X), ancestor(stannis_baratheon, X), child(X, Y)]
min depth: 3
Nodes: 83
Answers: 49

60: [child(_EX2, _EX6)]
min depth: 2
Nodes: 106
Answers: 104

61: [parent(X, yara_greyjoy), parent(X, theon_greyjoy)]
min depth: 1
Nodes: 3
Answers: 2

62: [son(X, doran_martell), child(X, mellario_of_norvos), ancestor(lewyn_martell, X)]
min depth: 3
Nodes: 50
Answers: 1

63: [parent(unknown_mother_targaryen, X), ancestor(unknown_mother_targaryen, X)]
min depth: 1
Nodes: 5
Answers: 4

64: [ancestor(tyros_lannister, X), ancestor(X, tyron_lannister)]
min depth: 2
Nodes: 43487
Answers: 12

65: [kill(X, Y), love(X, khal_drogo), who_betrayed_daenerys(Y)]
min depth: 1
Nodes: 8
Answers: 7

66: [descendant(nymeria_sand, X), ancestor(X, oberyn_martell)]
min depth: 3
Nodes: 517
Answers: 4

67: [ancestor(X, Y), parent(X, Z), parent(X, Y), status(Z, dead)]
Restart...
min depth: 2
Nodes: 100107
Answers: 127

68: [child(X, cassana_estermont)]
min depth: 2
Nodes: 5
Answers: 3

69: [descendant(X, _EX3)]
Restart...
Restart...
min depth: 3
Nodes: 23060
Answers: 168

70: [descendant(Z, lewyn_martell)]
min depth: 3
Nodes: 2305
Answers: 5

71: [ancestor(_EX3, sarella_sand)]
min depth: 2
Nodes: 15321
Answers: 2

72: [ancestor(unknown_mother_targaryen, X), ancestor(X, jon_snow), descendant(rhaeny_s_targaryen, X)]
min depth: 4
Nodes: 13924
Answers: 17

73: [daughter(_EX0, _EX4)]
min depth: 3

Nodes: 62
Answers: 28

74: [descendant(X, unknown_mother_martell), daughter(X, Y), son(Y, Z), descendant(X, Z), daughter(obara_sand, Y)]
min depth: 5
Nodes: 2448
Answers: 7

75: [descendant(X, steffon_baratheon), parent(Y, X), parent(cassana_estermont, Y)]
Restart...
Restart...
min depth: 5
Nodes: 1316
Answers: 7

76: [mother(unknown_mother_stark, _EX3)]
min depth: 3
Nodes: 10
Answers: 4

77: [child(rhaella_targaryen, Y)]
min depth: 2
Nodes: 4
Answers: 2

78: [love(X, Y), who_loved_daenerys(X), betray(viserys_targaryen, Y), love(hizdahr_zo_loraq, Y)]
min depth: 1
Nodes: 12
Answers: 11

79: [daughter(X, Y), descendant(rhaegar_targaryen, X), ancestor(X, Z), parent(Y, duncan_targaryen), ancestor(Y, Z)]
min depth: 3
Nodes: 134
Answers: 28

80: [status(X, alive), ancestor(Y, X), ancestor(Z, Y), male(Y), descendant(renly_baratheon, Z)]
min depth: 1
Nodes: 17
Answers: 16

81: [son(_EX3, _EX2)]
min depth: 3
Nodes: 178
Answers: 72

82: [descendant(_EX0, tytos_lannister)]
min depth: 3
Nodes: 1980
Answers: 11

83: [status(willem_lannister, X), status(Y, X), mother(unknown_mother_targaryen, Y)]
min depth: 1

Nodes: 2
Answers: 1

84: [descendant(Z, _EX5)]
min depth: 3
Nodes: 42086
Answers: 205

85: [descendant(martyn_lannister, Y)]
Restart...
min depth: 3
Nodes: 32810
Answers: 1

86: [daughter(Y, aerys_targaryen)]
min depth: 3
Nodes: 35
Answers: 1

87: [ancestor(_EX5, jaime_lannister)]
min depth: 2
Nodes: 486
Answers: 2

88: [descendant(rhaella_targaryen, X), parent(X, rhaella_targaryen)]
Restart...
Restart...
min depth: 3
Nodes: 24632
Answers: 2

89: [who_betrayed_daenerys(_EX1)]
min depth: 2
Nodes: 7
Answers: 5

90: [descendant(_EX6, alannys_harlaw)]
Restart...
min depth: 3
Nodes: 2647
Answers: 4

91: [son(_EX6, _EX1)]
min depth: 3
Nodes: 178
Answers: 72

92: [ancestor(_EX1, _EX5)]
min depth: 2
Nodes: 100115
Answers: 142

93: [son(X, Y), parent(Y, X), descendant(margaery_tyrell, Y)]
min depth: 3
Nodes: 121
Answers: 72

94: [descendant(X, Z), child(Y, X), descendant(Y, mellario_of_norvos), parent(Z, ella_martell)]
min depth: 3
Nodes: 42356
Answers: 153

95: [ancestor(X, Y), daughter(Y, X), female(Y)]
min depth: 2
Nodes: 1546
Answers: 198

96: [child(benjen_stark, X), ancestor(X, Y), descendant(bran_stark, Y)]
min depth: 2
Nodes: 4
Answers: 2

97: [descendant(sarella_sand, _EX4)]
Restart...
Restart...
min depth: 3
Nodes: 28315
Answers: 3

98: [mother(Y, benjen_stark)]
min depth: 3
Nodes: 5
Answers: 1

99: [ancestor(_EX0, joffery_lannister)]
min depth: 2
Nodes: 100166
Answers: 8

100: [ancestor(Z, _EX1)]
min depth: 2
Nodes: 24882
Answers: 246

Before removing duplicates: 1310606 examples
After removing duplicates: 21455 examples
Replicating pos examples 3 times
pos len=17305, neg len=17305

Train the scoring model. This step will usually take several minutes, especially if you are not using a GPU (for the gameofthrones KB, it takes about 30 minutes on CPU).

Creates a file:

- uni_mr_model.pt - Contains the weights for the learned scoring model

In [2]: !python nnreasoner.py --embed_type unification

States from vocab: vocab
Num predicates: 45
Max arity: 2
Num constants: 106
Num variables: 10

Training unification

Embed size: 50

Reading examples from mr_train_examples.csv

Parsing examples...

Generating embeddings...

Embedding input size: 277

[████████████████████] 34609/34609 Prepping examples

Loaded data...

Using cuda device

/opt/tljh/user/envs/cse327python311/lib/python3.11/site-packages/torch/nn/modules/lazy.py:180: UserWarning: Lazy modules are a new feature under heavy development so changes to the API or functionality can happen at any moment.

warnings.warn('Lazy modules are a new feature under heavy development '

0	0.61385 (-)
50	0.31681 (-0.0061)
100	0.27049 (-0.0036)
150	0.24313 (-0.0026)
200	0.22897 (-0.0020)
250	0.21627 (-0.0017)
300	0.20388 (-0.0014)
350	0.19261 (-0.0013)
400	0.18442 (-0.0011)
450	0.18001 (-0.0010)
500	0.17603 (-0.0009)
550	0.16999 (-0.0008)
600	0.16517 (-0.0008)
650	0.16248 (-0.0007)
700	0.16121 (-0.0006)
750	0.15829 (-0.0006)
800	0.15367 (-0.0006)
850	0.15261 (-0.0005)
900	0.15277 (-0.0005)
950	0.15092 (-0.0005)
1000	0.14815 (-0.0005)
1050	0.14684 (-0.0004)
1100	0.14726 (-0.0004)
1150	0.14460 (-0.0004)
1200	0.14280 (-0.0004)
1250	0.14234 (-0.0004)
1300	0.14264 (-0.0003)
1350	0.13991 (-0.0003)
1400	0.13861 (-0.0003)
1450	0.14086 (-0.0003)
1500	0.13797 (-0.0003)
1550	0.13569 (-0.0003)
1600	0.13513 (-0.0003)
1650	0.13353 (-0.0003)
1700	0.13339 (-0.0003)
1750	0.13432 (-0.0002)
1800	0.13186 (-0.0002)

```

1850    0.13361 (-0.0002)
1900    0.13055 (-0.0002)
1950    0.13220 (-0.0002)
2000    0.13164 (-0.0002)
2050    0.12945 (-0.0002)
2100    0.13042 (-0.0002)
2150    0.12909 (-0.0002)
2200    0.13022 (-0.0002)
2250    0.12955 (-0.0002)
2300    0.12668 (-0.0002)
2350    0.12733 (-0.0002)
2400    0.12563 (-0.0002)
2450    0.12704 (-0.0002)
2500    0.12562 (-0.0002)
2550    0.12448 (-0.0002)
2600    0.12562 (-0.0002)
2650    0.12436 (-0.0002)
2700    0.12359 (-0.0001)
2750    0.12417 (-0.0001)
2800    0.12196 (-0.0001)
2850    0.12249 (-0.0001)
2900    0.12341 (-0.0001)
2950    0.12289 (-0.0001)
3000    0.12142 (-0.0001)
3050    0.12086 (-0.0001)
3100    0.12212 (-0.0001)
3149    0.12075 (-0.0001)
Saved training loss figure.

```

```

In [4]: img = plt.imread('guided_loss-unification45-106-2-50.png')
        plt.imshow(img)

```

```

-----
NameError                                Traceback (most recent call last)
Cell In[4], line 1
----> 1 img = plt.imread('guided_loss-unification45-106-2-50.png')
      2 plt.imshow(img)

NameError: name 'plt' is not defined

```

Evaluate the scoring model by running evaluate.py. We compare the standard backward-chaining algorithm (-s) with the one that uses our learned scoring model (-u).

The output lists each query, and then provides a line that gives information about query execution. These are: number of nodes explored in the tree, depth of the solution, execution time in seconds, and the speed of exploring the tree (nodes per second).

The names of the files produced depend on the KB used and the options selected. The code below produces the files:

- std-45-106-2-50.csv - A CSV file with query results for the standard backward-chaining algorithm
- ming-45-106-2-50.csv - A CSV file with query results for the learned reasoner. ming stands for "min scoring goal" and is the default strategy for deciding how to make use

of the scores

If you use a different knowledge base, you will likely get different file names that depend on the features of that KB's vocabulary.

```
In [ ]: !python evaluate.py --kb gameofthrones.txt -s  
        !python evaluate.py --kb gameofthrones.txt -u
```

using cuda device
STANDARD

Query 1: [mother(X, kevan_lannister), parent(X, tywin_lannister)]
32 :: 4 - 00:00:00 (1592 nps)

Query 2: [male(X), kill(arya_stark, X), night_walkers_king(X), who_brought_the_winter(X), night_walker(X)]
33 :: 7 - 00:00:00 (16703 nps)

Query 3: [ancestor(Z, _EX1)]
3 :: 2 - 00:00:00 (1962 nps)

Query 4: [ancestor(lewyn_martell, _EX5)]
3 :: 2 - 00:00:00 (3073 nps)

Query 5: [descendant(X, _EX6)]
4 :: 3 - 00:00:00 (3333 nps)

Query 6: [descendant(_EX5, _EX6)]
4 :: 3 - 00:00:00 (3312 nps)

Query 7: [father(_EX2, maron_greyjoy)]
8 :: 3 - 00:00:00 (2171 nps)

Query 8: [child(X, rhaella_targaryen), descendant(X, rhaella_targaryen), descendant(X, Y), descendant(rhaella_targaryen, Y)]
59 :: 13 - 00:00:00 (3401 nps)

Query 9: [parent(elia_martell, X), male(X), child(X, elia_martell), ancestor(rhaegar_targaryen, X)]
7 :: 6 - 00:00:00 (2731 nps)

Query 10: [mother_dragons(daenerys_targaryen, X), status(X, Y), status(sansa_stark, Y), kill(night_king, X)]
8 :: 4 - 00:00:00 (3263 nps)

Query 11: [descendant(_EX0, Y)]
4 :: 3 - 00:00:00 (3286 nps)

Query 12: [ancestor(cassana_estermont, _EX4)]
3 :: 2 - 00:00:00 (3145 nps)

Query 13: [descendant(lancel_lannister, Z)]
4 :: 3 - 00:00:00 (5128 nps)

Query 14: [ancestor(_EX1, jon_snow)]
3 :: 2 - 00:00:00 (3770 nps)

Query 15: [male(X), child(X, Y), son(X, unknown_mother_targaryen), parent(Y, aerys_targaryen)]
26 :: 7 - 00:00:00 (189 nps)

Query 16: [ancestor(_EX0, tommen_lannister)]
3 :: 2 - 00:00:00 (3696 nps)

Query 17: [ancestor(_EX0, lyanna_stark)]
3 :: 2 - 00:00:00 (4084 nps)

Query 18: [daughter(_EX2, _EX3)]
6 :: 3 - 00:00:00 (2570 nps)

Query 19: [child(rhaella_targaryen, _EX2)]
3 :: 2 - 00:00:00 (3931 nps)

Query 20: [rightful_heir_iron_throne_descendant_from_last_king_or_queen(_EX5)]
8 :: 3 - 00:00:00 (1905 nps)

Query 21: [parent(Y, X), descendant(sansa_stark, X), ancestor(Y, rickon_stark)]
78 :: 8 - 00:00:00 (1966 nps)

Query 22: [descendant(jaime_lannister, _EX6)]
4 :: 3 - 00:00:00 (5180 nps)

Query 23: [descendant(_EX4, Y)]
4 :: 3 - 00:00:00 (2453 nps)

Query 24: [status(X, Z), father(Y, X), child(X, Y), status(lewyn_martell, Z), ancestor(Y, X)]
38 :: 9 - 00:00:00 (1435 nps)

Query 25: [daughter(sansa_stark, _EX1)]
4 :: 3 - 00:00:00 (4239 nps)

Query 26: [ancestor(_EX6, _EX3)]
3 :: 2 - 00:00:00 (2633 nps)

Query 27: [descendant(_EX6, X)]
4 :: 3 - 00:00:00 (3134 nps)

Query 28: [descendant(jon_snow, X)]
4 :: 3 - 00:00:00 (5350 nps)

Query 29: [descendant(Y, X), mother(X, Z), male(Y), child(Z, unknown_father_greyjoy), ancestor(alannys_harlaw, Y)]
1174 :: 13 - 00:00:00 (7602 nps)

Query 30: [daughter(X, catelyn_stark), father(Y, X), child(bran_stark, Y)]
22 :: 8 - 00:00:00 (1770 nps)

Query 31: [descendant(_EX0, unknown_mother_lannister)]
4 :: 3 - 00:00:00 (3919 nps)

Query 32: [ancestor(alannys_harlaw, _EX0)]
3 :: 2 - 00:00:00 (3261 nps)

Query 33: [parent(X, aerys_targaryen), father(X, rhaella_targaryen)]
5 :: 4 - 00:00:00 (3045 nps)

Query 34: [mother_dragons(X, viserion), love(X, khal_drogo)]
3 :: 2 - 00:00:00 (18121 nps)

Query 35: [daughter(X, _EX3)]
6 :: 3 - 00:00:00 (50279 nps)

Query 36: [ancestor(luthor_tyrell, mace_tyrell)]
3 :: 2 - 00:00:00 (4286 nps)

Query 37: [ancestor(_EX5, _EX1)]
3 :: 2 - 00:00:00 (2114 nps)

Query 38: [ancestor(X, rhaenys_targaryen), son(Y, X), son(Y, aerys_targaryen)]
147 :: 10 - 00:00:00 (1701 nps)

Query 39: [descendant(duncan_targaryen, unknown_mother_targaryen)]
4 :: 3 - 00:00:00 (4946 nps)

Query 40: [descendant(aegon_targaryen, Y)]
4 :: 3 - 00:00:00 (30419 nps)

Query 41: [mother(_EX1, sansa_stark)]
8 :: 3 - 00:00:00 (697 nps)

Query 42: [descendant(Z, _EX4)]
4 :: 3 - 00:00:00 (3275 nps)

Query 43: [son(X, alannys_harlaw), child(X, alannys_harlaw)]
29 :: 5 - 00:00:00 (1849 nps)

Query 44: [ancestor(_EX1, _EX0)]
3 :: 2 - 00:00:00 (1355 nps)

Query 45: [ancestor(unknown_mother_targaryen, X), parent(Y, X), ancestor(Y, aerys_targaryen)]
6 :: 5 - 00:00:00 (2240 nps)

Query 46: [ancestor(_EX6, trystane_martell)]
3 :: 2 - 00:00:00 (4009 nps)

Query 47: [rightful_heir_iron_throne_descendant_from_last_king_or_queen(_EX4)]
8 :: 3 - 00:00:00 (12212 nps)

Query 48: [descendant(_EX5, Y)]
4 :: 3 - 00:00:00 (2848 nps)

Query 49: [son(maron_greyjoy, Y)]
4 :: 3 - 00:00:00 (4055 nps)

Query 50: [child(Z, X)]
3 :: 2 - 00:00:00 (2434 nps)

Query 51: [descendant(Y, _EX4)]
4 :: 3 - 00:00:00 (3427 nps)

Query 52: [daughter(X, oberyn_martell), descendant(X, unknown_mother_sand_1)]
78 :: 6 - 00:00:00 (1844 nps)

Query 53: [descendant(rhaegar_targaryen, X), son(X, aegon_v_targaryen)]

7 :: 6 - 00:00:00 (3672 nps)

Query 54: [mother(catelyn_stark, _EX1)]

4 :: 3 - 00:00:00 (2645 nps)

Query 55: [child(_EX1, _EX3)]

3 :: 2 - 00:00:00 (2498 nps)

Query 56: [son(stannis_baratheon, X)]

4 :: 3 - 00:00:00 (3872 nps)

Query 57: [father(Y, X), ancestor(tyos_lannister, X), ancestor(Y, cersei_lannister)]

3095 :: 9 - 00:00:00 (3518 nps)

Query 58: [who_betrayed_daenerys(X), betray(X, Y), betray(varys, Y)]

5 :: 4 - 00:00:00 (15550 nps)

Query 59: [descendant(_EX0, tywin_lannister)]

4 :: 3 - 00:00:00 (3934 nps)

Query 60: [who_is_the_breaker_of_chains(_EX3)]

4 :: 3 - 00:00:00 (29002 nps)

Query 61: [son(maron_greyjoy, X), parent(X, Y), female(Y), son(theon_greyjoy, X)]

11 :: 8 - 00:00:00 (3766 nps)

Query 62: [child(_EX2, _EX0)]

3 :: 2 - 00:00:00 (1029 nps)

Query 63: [status(X, Y), father(X, Z), status(rickard_stark, Y), status(Z, Y)]

10 :: 6 - 00:00:00 (2415 nps)

Query 64: [ancestor(X, Y), son(Z, X), who_is_the_breaker_of_chains(Y), ancestor(X, Z), descendant(viserys_targaryen, Z)]

7315 :: 15 - 00:00:01 (3889 nps)

Query 65: [ancestor(_EX4, X)]

3 :: 2 - 00:00:00 (429 nps)

Query 66: [descendant(lancel_lannister, Z)]

4 :: 3 - 00:00:00 (43060 nps)

Query 67: [son(_EX0, _EX5)]

4 :: 3 - 00:00:00 (3242 nps)

Query 68: [child(Z, _EX0)]

3 :: 2 - 00:00:00 (2122 nps)

Query 69: [descendant(Y, X), father(X, balon_greyjoy), descendant(Y, Z), mother(Z, maron_greyjoy), descendant(yara_greyjoy, Z)]

Query failed!!!

3074 :: 0 - 00:00:00 (10048 nps)

Query 70: [who_loved_daenerys_and_was_corresponded(X), who_loved_daenerys(X), love(X, daenerys_targaryen)]

7 :: 6 - 00:00:00 (13481 nps)

Query 71: [descendant(X, lewyn_martell)]

4 :: 3 - 00:00:00 (29220 nps)

Query 72: [descendant(benjen_stark, _EX1)]

4 :: 3 - 00:00:00 (4686 nps)

Query 73: [ancestor(_EX3, _EX6)]

3 :: 2 - 00:00:00 (1985 nps)

Query 74: [son(X, rhaella_targaryen)]

32 :: 3 - 00:00:00 (1642 nps)

Query 75: [male(X), son(X, kevan_lannister), status(X, dead)]

60 :: 5 - 00:00:00 (2777 nps)

Query 76: [magic_ressurrected(X), creatures(X), status(X, alive)]

7 :: 4 - 00:00:00 (9687 nps)

Query 77: [mother(elia_martell, X), ancestor(Y, X), descendant(daenerys_targaryen, Y), female(Y)]

160 :: 11 - 00:00:00 (9486 nps)

Query 78: [descendant(_EX4, X)]

4 :: 3 - 00:00:00 (2299 nps)

Query 79: [child(_EX4, Z)]

3 :: 2 - 00:00:00 (2363 nps)

Query 80: [descendant(_EX4, Y)]

4 :: 3 - 00:00:00 (47959 nps)

Query 81: [destroy(Y, X), destroy(Z, X), kill(euron_greyjoy, Y), child(Z, aerys_targaryen)]

9 :: 5 - 00:00:00 (9129 nps)

Query 82: [descendant(trystane_martell, _EX3)]

4 :: 3 - 00:00:00 (4971 nps)

Query 83: [who_betrayed_daenerys(varys)]

3 :: 2 - 00:00:00 (24308 nps)

Query 84: [ancestor(X, Y), parent(X, rickon_stark), daughter(sansa_stark, X), son(Y, eddard_stark)]

18 :: 9 - 00:00:00 (3936 nps)

Query 85: [ancestor(_EX2, _EX1)]

3 :: 2 - 00:00:00 (170 nps)

Query 86: [descendant(Y, X), descendant(rhaenys_targaryen, X), ancestor(unknown_mother_targaryen, Y)]

1917 :: 12 - 00:00:00 (24440 nps)

Query 87: [ancestor(_EX5, _EX1)]

3 :: 2 - 00:00:00 (35506 nps)

Query 88: [father(Y, X), parent(cersei_lannister, X), descendant(Z, Y), descendant(Z, cersei_lannister)]
42 :: 10 - 00:00:00 (2642 nps)

Query 89: [take_away(drogon, X), love(Y, X), who_loved_daenerys_and_was_corresponded(Y)]
6 :: 5 - 00:00:00 (14860 nps)

Query 90: [mother(Z, rhaegar_targaryen)]
21 :: 3 - 00:00:00 (783 nps)

Query 91: [father(tywin_lannister, jaime_lannister)]
4 :: 3 - 00:00:00 (2948 nps)

Query 92: [descendant(loras_tyrell, X), descendant(margaery_tyrell, X)]
7 :: 6 - 00:00:00 (4594 nps)

Query 93: [descendant(rhaegar_targaryen, _EX1)]
4 :: 3 - 00:00:00 (5122 nps)

Query 94: [son(renly_baratheon, cassana_estermont)]
4 :: 3 - 00:00:00 (4068 nps)

Query 95: [descendant(_EX0, alannys_harlaw)]
4 :: 3 - 00:00:00 (4088 nps)

Query 96: [son(_EX4, _EX6)]
4 :: 3 - 00:00:00 (2292 nps)

Query 97: [who_betrayed_each_other(jaime_lannister, Z)]
4 :: 3 - 00:00:00 (17702 nps)

Query 98: [ancestor(_EX6, X)]
3 :: 2 - 00:00:00 (2065 nps)

Query 99: [son(aeron_greyjoy, X), descendant(aeron_greyjoy, X), descendant(balon_greyjoy, X)]
10 :: 9 - 00:00:00 (5359 nps)

Query 100: [father(jaime_lannister, X), son(X, jaime_lannister)]
7 :: 6 - 00:00:00 (5116 nps)

std: 178.13 nodes/query
1 queries failed
Time to run all queries: 3.9331447389999923
using cuda device
UNITY: ming
 Embedding Model: rKB_model.pth
 Scoring Model: uni_mr_model.pt

 Control: mingoal
 Rule Eval: Max Rule
Learned Embedding: Network using cuda
/opt/tljh/user/envs/cse327python311/lib/python3.11/site-packages/torch/nn/modules/lazy.py:180: UserWarning: Lazy modules are a new feature under heavy development so ch

anges to the API or functionality can happen at any moment.

```
warnings.warn('Lazy modules are a new feature under heavy development '
```

Query 1: [mother(X, kevan_lannister), parent(X, tywin_lannister)]

5 :: 4 - 00:00:00 (7 nps)

Query 2: [male(X), kill(arya_stark, X), night_walkers_king(X), who_brought_the_winter(X), night_walker(X)]

8 :: 7 - 00:00:00 (32 nps)

Query 3: [ancestor(Z, _EX1)]

3 :: 2 - 00:00:00 (5 nps)

Query 4: [ancestor(lewyn_martell, _EX5)]

3 :: 2 - 00:00:00 (93 nps)

Query 5: [descendant(X, _EX6)]

8 :: 5 - 00:00:01 (5 nps)

Query 6: [descendant(_EX5, _EX6)]

4 :: 3 - 00:00:00 (7 nps)

Query 7: [father(_EX2, maron_greyjoy)]

4 :: 3 - 00:00:00 (20 nps)

Query 8: [child(X, rhaella_targaryen), descendant(X, rhaella_targaryen), descendant(X, Y), descendant(rhaella_targaryen, Y)]

587 :: 13 - 00:00:17 (33 nps)

Query 9: [parent(elia_martell, X), male(X), child(X, elia_martell), ancestor(rhaegar_targaryen, X)]

7 :: 6 - 00:00:00 (32 nps)

Query 10: [mother_dragons(daenerys_targaryen, X), status(X, Y), status(sansa_stark, Y), kill(night_king, X)]

5 :: 4 - 00:00:00 (15 nps)

Query 11: [descendant(_EX0, Y)]

4 :: 3 - 00:00:00 (9 nps)

Query 12: [ancestor(cassana_estermont, _EX4)]

3 :: 2 - 00:00:00 (124 nps)

Query 13: [descendant(lancel_lannister, Z)]

4 :: 3 - 00:00:00 (211 nps)

Query 14: [ancestor(_EX1, jon_snow)]

3 :: 2 - 00:00:00 (143 nps)

Query 15: [male(X), child(X, Y), son(X, unknown_mother_targaryen), parent(Y, aerys_targaryen)]

14 :: 6 - 00:00:00 (115 nps)

Query 16: [ancestor(_EX0, tommen_lannister)]

3 :: 2 - 00:00:00 (144 nps)

Query 17: [ancestor(_EX0, lyanna_stark)]

3 :: 2 - 00:00:00 (150 nps)

Query 18: [daughter(_EX2, _EX3)]

5 :: 3 - 00:00:00 (8 nps)

Query 19: [child(rhaella_targaryen, _EX2)]

3 :: 2 - 00:00:00 (167 nps)

Query 20: [rightful_heir_iron_throne_descendant_from_last_king_or_queen(_EX5)]

4 :: 3 - 00:00:00 (40 nps)

Query 21: [parent(Y, X), descendant(sansa_stark, X), ancestor(Y, rickon_stark)]

Query failed!!!

1000000 :: 0 - 00:02:17 (7278 nps)

Query 22: [descendant(jaime_lannister, _EX6)]

4 :: 3 - 00:00:00 (124 nps)

Query 23: [descendant(_EX4, Y)]

4 :: 3 - 00:00:00 (7 nps)

Query 24: [status(X, Z), father(Y, X), child(X, Y), status(lewyn_martell, Z), ancestor(Y, X)]

8 :: 7 - 00:00:01 (4 nps)

Query 25: [daughter(sansa_stark, _EX1)]

4 :: 3 - 00:00:00 (177 nps)

Query 26: [ancestor(_EX6, _EX3)]

3 :: 2 - 00:00:00 (6 nps)

Query 27: [descendant(_EX6, X)]

Query failed!!!

1000000 :: 0 - 00:16:02 (1038 nps)

Query 28: [descendant(jon_snow, X)]

4 :: 3 - 00:00:00 (107 nps)

Query 29: [descendant(Y, X), mother(X, Z), male(Y), child(Z, unknown_father_greyjoy), ancestor(alannys_harlaw, Y)]

39 :: 13 - 00:00:00 (65 nps)

Query 30: [daughter(X, catelyn_stark), father(Y, X), child(bran_stark, Y)]

28 :: 8 - 00:00:00 (114 nps)

Query 31: [descendant(_EX0, unknown_mother_lannister)]

6 :: 5 - 00:00:00 (13 nps)

Query 32: [ancestor(alannys_harlaw, _EX0)]

3 :: 2 - 00:00:00 (116 nps)

Query 33: [parent(X, aerys_targaryen), father(X, rhaella_targaryen)]

8 :: 4 - 00:00:00 (195 nps)

Query 34: [mother_dragons(X, viserion), love(X, khal_drogo)]

3 :: 2 - 00:00:00 (262 nps)

Query 35: [daughter(X, _EX3)]
5 :: 3 - 00:00:00 (10 nps)

Query 36: [ancestor(luthor_tyrell, mace_tyrell)]
3 :: 2 - 00:00:00 (182 nps)

Query 37: [ancestor(_EX5, _EX1)]
3 :: 2 - 00:00:00 (7 nps)

Query 38: [ancestor(X, rhaenys_targaryen), son(Y, X), son(Y, aerys_targaryen)]
46 :: 7 - 00:00:01 (32 nps)

Query 39: [descendant(duncan_targaryen, unknown_mother_targaryen)]
286345 :: 3 - 00:00:27 (10534 nps)

Query 40: [descendant(aegon_targaryen, Y)]
4 :: 3 - 00:00:00 (143 nps)

Query 41: [mother(_EX1, sansa_stark)]
4 :: 3 - 00:00:00 (43 nps)

Query 42: [descendant(Z, _EX4)]
4 :: 3 - 00:00:00 (10 nps)

Query 43: [son(X, alannys_harlaw), child(X, alannys_harlaw)]
6 :: 5 - 00:00:00 (114 nps)

Query 44: [ancestor(_EX1, _EX0)]
3 :: 2 - 00:00:00 (6 nps)

Query 45: [ancestor(unknown_mother_targaryen, X), parent(Y, X), ancestor(Y, aerys_targaryen)]
6 :: 5 - 00:00:00 (12 nps)

Query 46: [ancestor(_EX6, trystane_martell)]
3 :: 2 - 00:00:00 (213 nps)

Query 47: [rightful_heir_iron_throne_descendant_from_last_king_or_queen(_EX4)]
4 :: 3 - 00:00:00 (71 nps)

Query 48: [descendant(_EX5, Y)]
4 :: 3 - 00:00:00 (13 nps)

Query 49: [son(maron_greyjoy, Y)]
4 :: 3 - 00:00:00 (72 nps)

Query 50: [child(Z, X)]
3 :: 2 - 00:00:00 (10 nps)

Query 51: [descendant(Y, _EX4)]
4 :: 3 - 00:00:00 (9 nps)

Query 52: [daughter(X, oberynt_martell), descendant(X, unknown_mother_sand_1)]
7 :: 6 - 00:00:00 (117 nps)

Query 53: [descendant(rhaegar_targaryen, X), son(X, aegon_v_targaryen)]
63772 :: 6 - 00:00:04 (13837 nps)

Query 54: [mother(catelyn_stark, _EX1)]
4 :: 3 - 00:00:00 (81 nps)

Query 55: [child(_EX1, _EX3)]
3 :: 2 - 00:00:00 (8 nps)

Query 56: [son(stannis_baratheon, X)]
4 :: 3 - 00:00:00 (195 nps)

Query 57: [father(Y, X), ancestor(tyos_lannister, X), ancestor(Y, cersei_lannister)]
20 :: 9 - 00:00:00 (99 nps)

Query 58: [who_betrayed_daenerys(X), betray(X, Y), betray(varys, Y)]
4 :: 3 - 00:00:00 (44 nps)

Query 59: [descendant(_EX0, tywin_lannister)]
4 :: 3 - 00:00:00 (143 nps)

Query 60: [who_is_the_breaker_of_chains(_EX3)]
4 :: 3 - 00:00:00 (185 nps)

Query 61: [son(maron_greyjoy, X), parent(X, Y), female(Y), son(theon_greyjoy, X)]
273 :: 8 - 00:00:01 (202 nps)

Query 62: [child(_EX2, _EX0)]
3 :: 2 - 00:00:00 (8 nps)

Query 63: [status(X, Y), father(X, Z), status(rickard_stark, Y), status(Z, Y)]
7 :: 6 - 00:00:01 (4 nps)

Query 64: [ancestor(X, Y), son(Z, X), who_is_the_breaker_of_chains(Y), ancestor(X, Z), descendant(viserys_targaryen, Z)]
1261 :: 14 - 00:00:18 (67 nps)

Query 65: [ancestor(_EX4, X)]
3 :: 2 - 00:00:00 (10 nps)

Query 66: [descendant(lancel_lannister, Z)]
4 :: 3 - 00:00:00 (223 nps)

Query 67: [son(_EX0, _EX5)]
5 :: 3 - 00:00:00 (12 nps)

Query 68: [child(Z, _EX0)]
3 :: 2 - 00:00:00 (9 nps)

Query 69: [descendant(Y, X), father(X, balon_greyjoy), descendant(Y, Z), mother(Z, maron_greyjoy), descendant(yara_greyjoy, Z)]
Query failed!!!
4363 :: 0 - 00:00:56 (77 nps)

Query 70: [who_loved_daenerys_and_was_corresponded(X), who_loved_daenerys(X), love

```

(X, daenerys_targaryen)]
5 :: 4 - 00:00:00 (63 nps)

Query 71: [descendant(X, lewyn_martell)]
4 :: 3 - 00:00:00 (171 nps)

Query 72: [descendant(benjen_stark, _EX1)]
4 :: 3 - 00:00:00 (223 nps)

Query 73: [ancestor(_EX3, _EX6)]
Query failed!!!
1000000 :: 0 - 00:17:08 (972 nps)

Query 74: [son(X, rhaella_targaryen)]
4 :: 3 - 00:00:00 (272 nps)

Query 75: [male(X), son(X, kevan_lannister), status(X, dead)]
5 :: 4 - 00:00:00 (21 nps)

Query 76: [magic_ressurrected(X), creatures(X), status(X, alive)]
9 :: 4 - 00:00:00 (204 nps)

Query 77: [mother(elia_martell, X), ancestor(Y, X), descendant(daenerys_targaryen,
Y), female(Y)]
1116 :: 11 - 00:00:03 (341 nps)

Query 78: [descendant(_EX4, X)]
4 :: 3 - 00:00:00 (30 nps)

Query 79: [child(_EX4, Z)]
3 :: 2 - 00:00:00 (24 nps)

Query 80: [descendant(_EX4, Y)]
4 :: 3 - 00:00:00 (19659 nps)

Query 81: [destroy(Y, X), destroy(Z, X), kill(euron_greyjoy, Y), child(Z, aerys_targ
aryen)]
15 :: 5 - 00:00:00 (437 nps)

Query 82: [descendant(trystane_martell, _EX3)]
4 :: 3 - 00:00:00 (452 nps)

Query 83: [who_betrayed_daenerys(varys)]
3 :: 2 - 00:00:00 (1081 nps)

Query 84: [ancestor(X, Y), parent(X, rickon_stark), daughter(sansa_stark, X), son(Y,
edward_stark)]
16 :: 9 - 00:00:00 (83 nps)

Query 85: [ancestor(_EX2, _EX1)]
3 :: 2 - 00:00:00 (23 nps)

Query 86: [descendant(Y, X), descendant(rhaenys_targaryen, X), ancestor(unknown_moth
er_targaryen, Y)]
34 :: 12 - 00:00:00 (396 nps)

```

Query 87: [ancestor(_EX5, _EX1)]
3 :: 2 - 00:00:00 (18956 nps)

Query 88: [father(Y, X), parent(cersei_lannister, X), descendant(Z, Y), descendant
(Z, cersei_lannister)]
14 :: 10 - 00:00:00 (44 nps)

Query 89: [take_away(drogon, X), love(Y, X), who_loved_daenerys_and_was_corresponded
(Y)]
6 :: 4 - 00:00:00 (111 nps)

Query 90: [mother(Z, rhaegar_targaryen)]
[-----] 0.0% to max depth (384 nps)

The results suggest only minor improvements for the gameofthrones KB: on average 3.75 vs. 4.39 node per query when I ran it. You may get different results due to the randomization of training and random selection of test queries. Larger, more complex KBs can see much larger improvements, sometimes an order of magnitude or more.

The code below shows the first 10 lines of each query result file. A given line has:

- query sequence number
- configuration name
- number of nodes explored
- depth of solution (0 if no solution found)
- did the system find a solution (true/false)
- time in seconds

You can import these files into a spreadsheet in order to create plots or summary statistics.

```
In [7]: !head std-45-106-2-50.csv  
!head ming-45-106-2-50.csv
```

```
query,std reasoner,std nodes explored,std min depth,success,time  
1,std,32,4,True,0.02009496600000027  
2,std,33,7,True,0.0019755799999998658  
3,std,3,2,True,0.0015283219999995268  
4,std,3,2,True,0.0009759840000000963  
5,std,4,3,True,0.0011998389999998693  
6,std,4,3,True,0.0012074389999998658  
7,std,8,3,True,0.0036835729999999955  
8,std,59,13,True,0.017347399000000152  
9,std,7,6,True,0.0025625079999999575  
query,ming reasoner,ming nodes explored,ming min depth,success,time  
1,ming,5,4,True,0.6581567860000001  
2,ming,8,7,True,0.24486948199999858  
3,ming,3,2,True,0.5300766029999995  
4,ming,3,2,True,0.032032372999999836  
5,ming,8,5,True,1.5601617079999999  
6,ming,4,3,True,0.5252669090000008  
7,ming,4,3,True,0.196291102  
8,ming,587,13,True,17.392180805  
9,ming,7,6,True,0.21597710900000067
```

STEP 5: Use the code to generate three random KBs of sizes 200, 300, and 400. To keep from overwriting important files, it is recommended that you create each in its own subdirectory. For example, if you want to put the 400 statement KB in a directory named size400: `cd size400 python ../kbencoder.py --generate_kb --num_rules 400 --new_vocab --save_vocab`

```
In [7]: %cd size300
!python ../kbencoder.py --generate_kb --num_rules 300 --new_vocab --save_vocab
```

```
[Errno 2] No such file or directory: 'size300'
/data/users/jupyter-zmf227/cse327-p4-files/size300
Classes: 1000000000.0
Max arity: 4
KB generated (300)
```

STEP 6: For each random KB, train the embedding and scoring models and then compare reasoning using the scoring model to the standard reasoner.

```
In [8]: # prepare and train using kbencoder
!python ../kbencoder.py --kb_path randomKB.txt --train_unification_model
```

Using cuda device

Uni model embed size: 50

```
[████████████████████] 3500/3500 Anchors generated
[████████████████████] 70000/70000 Triplets generated
[████████-----] 18401/70000 Triplet encodings
```

IOPub data rate exceeded.

The Jupyter server will temporarily stop sending output to the client in order to avoid crashing it.

To change this limit, set the config variable

`--ServerApp.iopub\_data\_rate\_limit`.

Current values:

ServerApp.iopub_data_rate_limit=1000000.0 (bytes/sec)

ServerApp.rate_limit_window=3.0 (secs)

```
[████████████████████-----] 46288/70000 Triplet encodings
```

IOPub data rate exceeded.

The Jupyter server will temporarily stop sending output to the client in order to avoid crashing it.

To change this limit, set the config variable

`--ServerApp.iopub\_data\_rate\_limit`.

Current values:

ServerApp.iopub_data_rate_limit=1000000.0 (bytes/sec)

ServerApp.rate_limit_window=3.0 (secs)

```
[████████████████████] 70000/70000 Triplet encodings
```

```
Early stopping triggered-----] 20.2% Epoch | [Epoch 102/500, Training Loss: 0.2260, Validation Loss: 0.4858]
```

Saved model.

Saved plot.

unification model generated

Generate a set of train and test queries from your knowledge base. This will also generate the deductive closure (all facts that can be inferred) of your KB using forward-chaining.

Produces the files:

all_facts.txt - A list of all facts that can be inferred from the KB. This is used to generate queries that are guaranteed to have an answer. train_queries.txt - A set of training queries where each will have at least one answer. test_queries.txt - A set of test queries where each will have at least one answer.

```
In [9]: !python ../kbencoder.py --kb_path randomKB.txt -g
```

```
Generating facts list...
Forward chaining depth: 1
New facts: 78
Forward chaining depth: 2
New facts: 24
Forward chaining depth: 3

Facts list generated
102 total facts. Max depth = 2

Number of queries: 200
100 training queries generated
100 testing queries generated
```

Create a training set for the scoring model. The examples will consist of goal, rule, score pairs, where a score of 1 means the rule can solve the goal, and 0 means it cannot. It runs the full set of training queries with a randomized backward-chaining process. This step can sometimes take a long time and is not able to take advantage of a GPU (with the gameofthrones KB it took ~45 minutes).

Creates a file:

- mr_train_examples.csv - Contains a line for each goal/rule/score example.

```
In [10]: !python ../kbencoder.py --kb_path randomKB.txt -p
```

running training queries...

Negative facts will be generated

1: [p6(X4, X9)]

min depth: 1

Nodes: 47

Answers: 44

2: [p5(X0, a4, a55), p1(X2, X0, X1, X2), p1(a71, X0, X1, a71), p1(a1, X0, X1, X0), p9(a1, X2)]

min depth: 1

Nodes: 2

Answers: 1

3: [p1(a77, X5, X4, X8)]

min depth: 2

Nodes: 66

Answers: 2

4: [p0(a92, X0), p1(a24, X1, a11, X0), p1(X1, a0, a5, X1)]

min depth: 1

Nodes: 2

Answers: 1

5: [p8(X0, X0), p4(X0, a70)]

min depth: 2

Nodes: 48

Answers: 26

6: [p5(X0, a75, X1), p3(X0), p9(X1, X2), p1(a95, a0, a5, X2)]

min depth: 1

Nodes: 2

Answers: 1

7: [p9(X0, X0), p4(a13, X0)]

min depth: 1

Nodes: 29

Answers: 27

8: [p9(X9, X9)]

min depth: 1

Nodes: 29

Answers: 27

9: [p2(X0, a68, a14), p2(a47, a0, X0)]

min depth: 1

Nodes: 2

Answers: 1

10: [p9(a11, a11)]

min depth: 2

Nodes: 3

Answers: 1

11: [p6(a0, X0)]

min depth: 1

Nodes: 11

Answers: 8

12: [p9(X2, X2)]

min depth: 1

Nodes: 29

Answers: 27

13: [p9(X7, X7)]

min depth: 2

Nodes: 29

Answers: 27

14: [p8(X5, X5)]

min depth: 2

Nodes: 29

Answers: 26

15: [p1(X0, a0, X5, X0)]

min depth: 3

Nodes: 30

Answers: 26

16: [p1(X4, X6, X1, X4)]

min depth: 1

Nodes: 312

Answers: 1

17: [p6(X4, X5)]

min depth: 1

Nodes: 72

Answers: 44

18: [p4(X0, a26), p4(X0, a45), p4(X0, X1), p1(a66, X1, a32, a2), p5(a25, a70, X0)]

min depth: 1

Nodes: 2

Answers: 1

19: [p1(a58, X8, X0, a11)]

min depth: 2

Nodes: 4

Answers: 1

20: [p1(X1, X1, X6, X9)]

min depth: 2

Nodes: 491

Answers: 8

21: [p6(X0, a73), p5(X1, a40, X0), p3(X1), p4(a61, X0)]

[-----] 0.0% to max nodes (42813 nps)Mismatched predicates: p5(X1, a40, X0) with p4(a73, X0)

min depth: 1

Nodes: 3

Answers: 1

22: [p1(X8, X7, a5, a91)]

min depth: 2

Nodes: 20
Answers: 1

23: [p8(X0, X0)]
min depth: 2
Nodes: 151
Answers: 26

24: [p1(a77, X4, X0, X7)]
min depth: 3
Nodes: 7
Answers: 3

25: [p6(X9, X7)]
min depth: 1
Nodes: 69
Answers: 41

26: [p6(X0, X1), p6(X0, a5), p8(X1, X1), p7(X0, a32)]
min depth: 1
Nodes: 72
Answers: 44

27: [p1(X8, a0, X5, a53)]
min depth: 2
Nodes: 13
Answers: 2

28: [p7(X0, X1)]
min depth: 1
Nodes: 820
Answers: 30

29: [p7(a80, X0)]
min depth: 1
Nodes: 159
Answers: 1

30: [p6(X0, a16), p7(X0, a32), p6(X0, a5)]
min depth: 3
Nodes: 6
Answers: 3

31: [p1(X4, X9, a5, a28)]
min depth: 2
Nodes: 6
Answers: 2

32: [p8(X6, X6)]
min depth: 2
Nodes: 301
Answers: 26

33: [p8(X8, X8)]
min depth: 2
Nodes: 507

Answers: 26

34: [p3(X5)]
min depth: 2
Nodes: 1834
Answers: 773

35: [p9(X7, X7)]
min depth: 2
Nodes: 29
Answers: 27

36: [p7(a17, X0), p5(a24, X1, X0), p3(X1), p5(X1, a11, a4)]
min depth: 1
Mismatched predicates: p5(a24, X1, X0) with p2(a16, a17, X0)
Nodes: 74
Answers: 2

37: [p1(a58, X4, X7, X5)]
min depth: 2
Nodes: 5
Answers: 1

38: [p1(X7, X4, a5, X0)]
min depth: 2
Nodes: 1049
Answers: 82

39: [p1(a93, X2, X3, X5)]
min depth: 2
Nodes: 67
Answers: 2

40: [p1(a22, a0, X2, a11)]
min depth: 2
Nodes: 8
Answers: 1

41: [p1(a10, X0, a47, a4), p7(a13, X0)]
min depth: 1
Nodes: 2
Answers: 1

42: [p1(X0, a21, X2, a13), p1(a10, X0, a47, X1), p9(a69, X1), p1(a93, a0, X3, X2), p1(a0, a0, X3, X2)]
min depth: 1
Nodes: 2
Answers: 1

43: [p1(X4, X2, X7, a29)]
Restart...
min depth: 2
Nodes: 5
Answers: 1

44: [p8(X0, X0), p0(X0, a0), p0(X0, X1), p5(X0, a11, X1)]

min depth: 2
Mismatched predicates: $p_0(X_0, a_0)$ with $p_2(X_0, a_{48}, X_0)$
Nodes: 95
Answers: 26

45: $[p_1(X_8, X_2, a_5, X_8)]$
min depth: 3
Nodes: 31
Answers: 26

46: $[p_9(X_8, X_8)]$
min depth: 1
Nodes: 29
Answers: 27

47: $[p_1(a_{39}, a_0, a_5, X_0), p_8(a_{39}, X_0)]$
min depth: 2
Nodes: 5
Answers: 1

48: $[p_8(X_5, X_5)]$
min depth: 2
Nodes: 269
Answers: 26

49: $[p_1(X_1, a_0, a_5, a_{49})]$
min depth: 2
Nodes: 6
Answers: 1

50: $[p_4(a_{91}, X_0), p_7(X_0, X_1), p_1(a_{66}, X_1, X_2, a_2), p_1(a_{52}, X_2, a_{78}, X_3), p_8(X_3, X_3)]$
min depth: 1
Nodes: 2
Answers: 1

51: $[p_1(X_2, a_0, a_5, a_0)]$
min depth: 3
Nodes: 9
Answers: 5

52: $[p_1(a_{78}, X_7, a_5, a_{19})]$
min depth: 2
Nodes: 15
Answers: 1

53: $[p_5(X_0, a_0, a_{59}), p_5(a_1, a_{69}, X_0), p_1(a_{63}, a_{46}, X_0, a_{84})]$
min depth: 1
Nodes: 2
Answers: 1

54: $[p_6(a_0, X_1)]$
min depth: 1
Nodes: 11
Answers: 8

55: $[p_8(X_0, X_0)]$

min depth: 2

Nodes: 30

Answers: 26

56: [p1(a11, X3, X0, X4)]

min depth: 2

Nodes: 5

Answers: 1

57: [p1(X1, X0, X1, a33), p7(X0, a84), p1(a93, X1, a5, X2), p7(a21, X1), p2(X2, a22, a10)]

min depth: 1

Nodes: 4

Answers: 1

58: [p1(a0, a0, X1, X3)]

min depth: 2

Nodes: 75

Answers: 9

59: [p1(X0, X2, a5, X1), p1(X0, X2, a5, X0), p5(X2, X3, X1), p4(X2, X2), p0(X3, a97)]

min depth: 2

Nodes: 63

Answers: 58

60: [p7(X0, a79), p8(X1, X0), p1(X1, a0, a5, a1), p5(a30, X0, a23), p8(X1, a1)]

min depth: 1

Nodes: 195

Answers: 1

61: [p9(X1, X1)]

min depth: 1

Nodes: 29

Answers: 27

62: [p1(a95, a0, X2, X1)]

min depth: 2

Nodes: 7

Answers: 3

63: [p0(a90, X0), p9(X1, X0), p8(a0, X1)]

min depth: 1

Nodes: 3

Answers: 2

64: [p2(a12, a72, X0), p0(X0, a50)]

min depth: 1

Nodes: 2

Answers: 1

65: [p6(X3, X3)]

min depth: 3

Nodes: 6

Answers: 2

66: [p1(a78, X5, X4, a19)]

min depth: 2

Nodes: 4

Answers: 1

67: [p1(a22, X1, X0, a22), p1(a16, X1, X0, a16), p9(a6, X1), p4(X1, X1)]

min depth: 3

Nodes: 23

Answers: 1

68: [p1(X0, a0, a5, X0)]

min depth: 3

Nodes: 31

Answers: 26

69: [p6(X8, a18)]

min depth: 3

Nodes: 4

Answers: 1

70: [p9(X3, X3)]

min depth: 1

Nodes: 29

Answers: 27

71: [p0(X0, a11), p5(X1, a0, X0), p0(X2, X1), p9(a1, X2)]

min depth: 1

Nodes: 3

Answers: 2

72: [p1(a1, a0, X8, a53)]

min depth: 2

Nodes: 4

Answers: 1

73: [p9(X6, X6)]

min depth: 1

Nodes: 29

Answers: 27

74: [p8(X0, a19), p1(X0, a0, a5, a19)]

[-----] 0.0% to max nodes (288933 nps)Mismatched predicates: p1(X0, a0, a5, a19) with p4(a19, X20)

Mismatched predicates: p1(X0, a0, a5, a19) with p4(a19, X20)

Restart...

[-----] 0.0% to max nodes (165316 nps)Mismatched predicates: p1(X0, a0, a5, a19) with p4(a19, X20)

min depth: 1

Nodes: 5

Answers: 1

75: [p2(a9, X1, X0), p6(X0, a14), p1(X1, X2, a5, X1), p2(a95, a23, X2)]

min depth: 1

Nodes: 2

Answers: 1

76: [p9(a42, a42)]

min depth: 2

Nodes: 3

Answers: 1

77: [p1(a1, a0, X7, X8)]

min depth: 2

Nodes: 6

Answers: 2

78: [p2(a24, a12, X0), p5(a26, a46, X0)]

min depth: 1

Nodes: 2

Answers: 1

79: [p6(X0, a5), p6(X0, X1), p7(X0, a32), p6(a99, X1)]

min depth: 1

Nodes: 5

Answers: 2

80: [p1(a52, a32, X0, a3), p8(X0, a19), p0(a30, X0)]

min depth: 1

Nodes: 2

Answers: 1

81: [p1(a66, a86, X0, X1), p1(a52, X0, X3, a3), p1(a63, X2, X1, a84), p7(X2, a11), p

8(X3, a19)]

min depth: 1

Nodes: 2

Answers: 1

82: [p9(X1, X1)]

min depth: 2

Nodes: 29

Answers: 27

83: [p2(X0, a70, a89), p8(X0, a67)]

min depth: 1

Nodes: 2

Answers: 1

84: [p6(X8, X8)]

min depth: 3

Nodes: 6

Answers: 2

85: [p1(X3, X9, a5, X3)]

Restart...

min depth: 3

Nodes: 195

Answers: 26

86: [p9(X0, X0), p4(a43, X0), p9(X0, X0), p5(a0, a25, X0)]

min depth: 1

Nodes: 29

Answers: 27

87: [p0(a34, X0), p8(a78, X0), p8(X0, a67)]

min depth: 1

Nodes: 2

Answers: 1

88: [p1(a63, X0, a2, X1), p5(a26, X0, a8), p1(a71, a0, a5, X1)]

min depth: 1

Nodes: 2

Answers: 1

89: [p9(X0, a0)]

min depth: 1

Nodes: 16

Answers: 14

90: [p2(X0, a68, a14), p2(X1, a0, X0), p1(a10, a29, X1, a4)]

min depth: 1

Nodes: 2

Answers: 1

91: [p9(X1, X1)]

min depth: 2

Nodes: 29

Answers: 27

92: [p4(X0, a86), p4(X0, X1), p5(X2, a25, X1), p1(a12, X2, a5, a53)]

min depth: 1

Nodes: 2

Answers: 1

93: [p1(X2, a0, X4, X2)]

min depth: 3

Nodes: 30

Answers: 26

94: [p1(a6, a0, X0, a1), p1(a0, a0, X0, X1), p8(a0, X1)]

min depth: 2

Nodes: 4

Answers: 1

95: [p8(X1, X1)]

min depth: 2

Nodes: 29

Answers: 26

96: [p1(a3, a0, X7, X1)]

min depth: 3

Nodes: 6

Answers: 2

97: [p1(a78, X9, X7, X0)]

Restart...

min depth: 2

Nodes: 65

Answers: 1

98: [p1(X0, a0, X3, X0)]

min depth: 3

Nodes: 30

Answers: 26

99: [p1(X0, X1, a5, X0), p8(a11, X0), p8(a75, X1), p6(X0, a14)]

min depth: 3

Nodes: 30

Answers: 26

100: [p1(a93, a0, X1, X0), p2(X0, a66, a99), p1(a75, a0, X1, a0)]

min depth: 2

Nodes: 6

Answers: 2

Before removing duplicates: 10500 examples

After removing duplicates: 2645 examples

pos len=1474, neg len=1474

In [11]: !python ../nnreasoner.py --embed_type unification

States from vocab: vocab
Num predicates: 10
Max arity: 4
Num constants: 100
Num variables: 10

Training unification

Embed size: 50

Reading examples from mr_train_examples.csv

Parsing examples...

Generating embeddings...

Embedding input size: 450

[████████████████] 2947/2947 Prepping examples

Loaded data...

Using cuda device

/opt/tljh/user/envs/cse327python311/lib/python3.11/site-packages/torch/nn/modules/lazy.py:180: UserWarning: Lazy modules are a new feature under heavy development so changes to the API or functionality can happen at any moment.

warnings.warn('Lazy modules are a new feature under heavy development '

0	0.61625 (-)
50	0.22428 (-0.0076)
100	0.20698 (-0.0037)
150	0.19568 (-0.0026)
200	0.18519 (-0.0020)
250	0.17372 (-0.0017)
300	0.16755 (-0.0014)
350	0.15981 (-0.0012)
400	0.15588 (-0.0011)
450	0.15175 (-0.0009)
500	0.14509 (-0.0009)
550	0.14375 (-0.0008)
600	0.13696 (-0.0007)
650	0.13425 (-0.0007)
700	0.12903 (-0.0006)
750	0.12479 (-0.0006)
800	0.12296 (-0.0006)
850	0.11791 (-0.0005)
900	0.11427 (-0.0005)
950	0.11443 (-0.0005)
1000	0.10618 (-0.0004)
1050	0.10804 (-0.0004)
1100	0.10055 (-0.0004)
1150	0.09522 (-0.0004)
1200	0.09355 (-0.0004)
1250	0.09012 (-0.0004)
1300	0.09140 (-0.0003)
1350	0.08830 (-0.0003)
1400	0.08406 (-0.0003)
1450	0.08287 (-0.0003)
1500	0.08064 (-0.0003)
1550	0.07858 (-0.0003)
1600	0.07726 (-0.0003)
1650	0.08024 (-0.0003)
1700	0.07673 (-0.0003)
1750	0.07515 (-0.0003)
1800	0.07322 (-0.0002)

1850	0.07194	(-0.0002)
1900	0.07322	(-0.0002)
1950	0.06976	(-0.0002)
2000	0.07084	(-0.0002)
2050	0.06915	(-0.0002)
2100	0.06800	(-0.0002)
2150	0.06566	(-0.0002)
2200	0.06715	(-0.0002)
2250	0.06305	(-0.0002)
2300	0.06681	(-0.0002)
2350	0.06570	(-0.0002)
2400	0.06132	(-0.0002)
2450	0.06279	(-0.0002)
2500	0.06067	(-0.0002)
2550	0.06066	(-0.0002)
2600	0.05839	(-0.0002)
2650	0.05936	(-0.0002)
2700	0.06245	(-0.0002)
2750	0.05779	(-0.0001)
2800	0.05910	(-0.0001)
2849	0.06050	(-0.0001)

Saved training loss figure.

Evaluate the scoring model by running `evaluate.py`. We compare the standard backward-chaining algorithm (-s) with the one that uses our learned scoring model (-u).

The output lists each query, and then provides a line that gives information about query execution. These are: number of nodes explored in the tree, depth of the solution, execution time in seconds, and the speed of exploring the tree (nodes per second).

The names of the files produced depend on the KB used and the options selected. The code below produces the files:

- `std-45-106-2-50.csv` - A CSV file with query results for the standard backward-chaining algorithm
- `ming-45-106-2-50.csv` - A CSV file with query results for the learned reasoner. `ming` stands for "min scoring goal" and is the default strategy for deciding how to make use of the scores

If you use a different knowledge base, you will likely get different file names that depend on the features of that KB's vocabulary.

```
In [12]: # evaluate and compare
!python ../evaluate.py --kb randomKB.txt -s
!python ../evaluate.py --kb randomKB.txt -u
```

using cuda device
STANDARD

Query 1: [p2(a12, a72, X0), p0(X0, X1), p3(X1), p8(a63, X1)]
5 :: 4 - 00:00:00 (3960 nps)

Query 2: [p1(X1, a0, a5, X0)]
3 :: 2 - 00:00:00 (4114 nps)

Query 3: [p0(X1, X0), p6(a37, X0), p8(a77, X1)]
78 :: 3 - 00:00:00 (7886 nps)

Query 4: [p1(X7, a0, a5, X7)]
4 :: 3 - 00:00:00 (4426 nps)

Query 5: [p1(X1, a0, a5, X1)]
4 :: 3 - 00:00:00 (4452 nps)

Query 6: [p1(X2, a0, a5, X6)]
3 :: 2 - 00:00:00 (4389 nps)

Query 7: [p6(X0, a18)]
4 :: 3 - 00:00:00 (3575 nps)

Query 8: [p2(X0, X1, a92), p6(a94, X0), p5(a0, X1, a39)]
4 :: 3 - 00:00:00 (4666 nps)

Query 9: [p8(X3, X3)]
3 :: 2 - 00:00:00 (4652 nps)

Query 10: [p5(a20, X0, a24), p1(a0, X0, a14, a10)]
3 :: 2 - 00:00:00 (6147 nps)

Query 11: [p1(X6, X1, X7, X6)]
2 :: 1 - 00:00:00 (5563 nps)

Query 12: [p9(X5, X5)]
2 :: 1 - 00:00:00 (9054 nps)

Query 13: [p8(X1, X1)]
3 :: 2 - 00:00:00 (5023 nps)

Query 14: [p4(X0, X1), p8(X0, X0), p2(a14, X1, X2), p8(X2, X2), p0(a0, X2)]
102 :: 7 - 00:00:00 (5854 nps)

Query 15: [p8(X0, X0), p1(X0, a0, a5, X0)]
6 :: 5 - 00:00:00 (3478 nps)

Query 16: [p9(a27, a27)]
3 :: 2 - 00:00:00 (8551 nps)

Query 17: [p3(X0), p1(a41, a58, a10, X0), p9(X0, X0), p2(a75, a54, X0)]
58 :: 11 - 00:00:00 (5199 nps)

Query 18: [p1(a1, X6, a5, X6)]
3 :: 2 - 00:00:00 (5287 nps)

Query 19: [p1(a17, X1, X3, a17)]

4 :: 3 - 00:00:00 (5853 nps)

Query 20: [p5(X0, a10, X1), p2(a40, X0, a11), p9(X1, X1)]

5 :: 4 - 00:00:00 (3958 nps)

Query 21: [p1(X6, a0, X5, X6)]

4 :: 3 - 00:00:00 (4269 nps)

Query 22: [p3(X7)]

2 :: 1 - 00:00:00 (7922 nps)

Query 23: [p2(a98, X1, X0), p4(X0, a70), p7(X1, a35)]

4 :: 3 - 00:00:00 (4316 nps)

Query 24: [p9(a27, a27)]

3 :: 2 - 00:00:00 (53800 nps)

Query 25: [p1(a11, a0, a5, a31)]

3 :: 2 - 00:00:00 (6502 nps)

Query 26: [p8(a0, X0), p8(X0, X0)]

13 :: 3 - 00:00:00 (5579 nps)

Query 27: [p9(a6, a6)]

3 :: 2 - 00:00:00 (7730 nps)

Query 28: [p0(X0, a0), p2(X0, X2, X1), p7(a79, X1), p8(X2, X2)]

8 :: 5 - 00:00:00 (3784 nps)

Query 29: [p9(X4, X4)]

2 :: 1 - 00:00:00 (8691 nps)

Query 30: [p9(a38, X0), p1(a91, X0, a5, a91), p1(a25, X0, a23, a54)]

6 :: 5 - 00:00:00 (3899 nps)

Query 31: [p1(X1, a0, X0, X1)]

4 :: 3 - 00:00:00 (9702 nps)

Query 32: [p1(X0, X3, a5, X9)]

3 :: 2 - 00:00:00 (4307 nps)

Query 33: [p1(X8, X8, a5, a77)]

3 :: 2 - 00:00:00 (6622 nps)

Query 34: [p6(X9, X9)]

4 :: 3 - 00:00:00 (6024 nps)

Query 35: [p5(a58, a1, X0), p8(X0, X0), p8(a93, X0)]

5 :: 4 - 00:00:00 (7753 nps)

Query 36: [p1(a1, X9, X1, X9)]

3 :: 2 - 00:00:00 (8867 nps)

Query 37: [p1(a78, a0, X0, a19), p1(a12, a0, X0, a53)]

5 :: 4 - 00:00:00 (5478 nps)

Query 38: [p1(a6, a0, X2, X9)]

3 :: 2 - 00:00:00 (4596 nps)

Query 39: [p3(X4)]

2 :: 1 - 00:00:00 (7648 nps)

Query 40: [p8(X3, X3)]

3 :: 2 - 00:00:00 (48798 nps)

Query 41: [p5(a20, a74, X0), p1(X0, a43, a11, a50)]

3 :: 2 - 00:00:00 (5256 nps)

Query 42: [p8(a19, X0), p1(X0, a0, X1, a0), p1(a19, a0, X1, X0), p4(X1, a48)]

7 :: 6 - 00:00:00 (4899 nps)

Query 43: [p1(a71, X3, a5, X2)]

3 :: 2 - 00:00:00 (4678 nps)

Query 44: [p0(X1, X0), p3(X1)]

212 :: 2 - 00:00:00 (1384 nps)

Query 45: [p9(X4, X4)]

2 :: 1 - 00:00:00 (28587 nps)

Query 46: [p6(X5, X0)]

2 :: 1 - 00:00:00 (3836 nps)

Query 47: [p9(X9, X9)]

2 :: 1 - 00:00:00 (8170 nps)

Query 48: [p8(X2, X0), p7(X0, X1), p6(a33, X1), p5(a27, X2, a66)]

1040 :: 4 - 00:00:00 (7968 nps)

Query 49: [p1(a63, X1, a5, X0), p5(a13, a11, X0), p8(a1, X1)]

5 :: 4 - 00:00:00 (4504 nps)

Query 50: [p1(a93, X1, X2, X5)]

3 :: 2 - 00:00:00 (4385 nps)

Query 51: [p9(X6, X6)]

2 :: 1 - 00:00:00 (8701 nps)

Query 52: [p6(a0, X0), p9(X0, a0), p1(X0, a0, a5, a13)]

9 :: 6 - 00:00:00 (5689 nps)

Query 53: [p9(X8, X8)]

2 :: 1 - 00:00:00 (8651 nps)

Query 54: [p0(X1, X0), p9(X0, X0), p5(X2, a0, X1), p5(a1, X3, X2), p4(a17, X3)]

69 :: 6 - 00:00:00 (7635 nps)

Query 55: [p7(X0, X1)]

2 :: 1 - 00:00:00 (5131 nps)

Query 56: [p7(a22, X0), p2(X0, a70, a15)]

3 :: 2 - 00:00:00 (4923 nps)

Query 57: [p9(X9, X9)]

2 :: 1 - 00:00:00 (41742 nps)

Query 58: [p1(a61, a0, X3, X7)]

3 :: 2 - 00:00:00 (4617 nps)

Query 59: [p6(X7, a20)]

2 :: 1 - 00:00:00 (6423 nps)

Query 60: [p0(X0, X1), p7(X0, X2), p8(X1, a19), p2(a19, X3, X2), p2(a57, X3, a15)]

1192 :: 5 - 00:00:00 (8957 nps)

Query 61: [p1(X0, a0, X4, X0)]

4 :: 3 - 00:00:00 (4272 nps)

Query 62: [p1(X3, a0, X5, X3)]

4 :: 3 - 00:00:00 (4388 nps)

Query 63: [p1(X3, a0, X9, X3)]

4 :: 3 - 00:00:00 (10939 nps)

Query 64: [p1(X6, X5, a5, X6)]

4 :: 3 - 00:00:00 (13076 nps)

Query 65: [p6(X0, a17), p2(X1, X0, a35), p7(X0, a0), p0(X1, a0), p8(X0, X0)]

7 :: 6 - 00:00:00 (4402 nps)

Query 66: [p8(X8, X8)]

3 :: 2 - 00:00:00 (4996 nps)

Query 67: [p1(a71, a0, X7, X1)]

3 :: 2 - 00:00:00 (4756 nps)

Query 68: [p2(X0, a1, a49), p1(X0, a0, a5, X0)]

5 :: 4 - 00:00:00 (5249 nps)

Query 69: [p2(X0, a54, a0), p5(X0, a22, a16), p4(a22, X0)]

4 :: 3 - 00:00:00 (5455 nps)

Query 70: [p6(X1, X0), p4(X0, a75), p9(X0, X0), p5(a41, X1, a0)]

27 :: 5 - 00:00:00 (7488 nps)

Query 71: [p0(X0, a4), p5(X0, a11, a50)]

3 :: 2 - 00:00:00 (5240 nps)

Query 72: [p8(X9, X9)]

3 :: 2 - 00:00:00 (4676 nps)

Query 73: [p0(X0, a88), p5(X0, a40, a27)]

3 :: 2 - 00:00:00 (5685 nps)

Query 74: [p1(X0, X1, a5, X0), p2(X1, a14, X0), p1(a77, X1, a5, a10), p9(a91, X1)]

33 :: 7 - 00:00:00 (5071 nps)

Query 75: [p1(a22, X2, a5, a11)]
3 :: 2 - 00:00:00 (6376 nps)

Query 76: [p1(a71, a0, a5, X7)]
3 :: 2 - 00:00:00 (5036 nps)

Query 77: [p1(a0, a0, a5, X6)]
3 :: 2 - 00:00:00 (5113 nps)

Query 78: [p0(X0, X1), p9(X0, X0), p3(X0), p6(X0, a1), p2(X1, a87, a11)]
19040 :: 6 - 00:00:00 (59020 nps)

Query 79: [p1(X0, X1, a5, X0), p2(a44, a20, X0), p8(X1, X1)]
10 :: 6 - 00:00:00 (5918 nps)

Query 80: [p6(a0, X0)]
2 :: 1 - 00:00:00 (34930 nps)

Query 81: [p6(a0, a18)]
4 :: 3 - 00:00:00 (11811 nps)

Query 82: [p6(X0, a16), p2(a48, X0, a34)]
4 :: 2 - 00:00:00 (4757 nps)

Query 83: [p1(X0, X5, X3, X1)]
2 :: 1 - 00:00:00 (4762 nps)

Query 84: [p9(X3, X3)]
2 :: 1 - 00:00:00 (8498 nps)

Query 85: [p8(X5, X5)]
3 :: 2 - 00:00:00 (4738 nps)

Query 86: [p1(X0, X4, X6, X0)]
2 :: 1 - 00:00:00 (5329 nps)

Query 87: [p6(X8, X5)]
2 :: 1 - 00:00:00 (4407 nps)

Query 88: [p1(a58, a0, X2, a11)]
3 :: 2 - 00:00:00 (6267 nps)

Query 89: [p2(X0, a20, a10), p6(X0, a0)]
3 :: 2 - 00:00:00 (4988 nps)

Query 90: [p5(a87, X0, a30), p7(X0, a94)]
3 :: 2 - 00:00:00 (5448 nps)

Query 91: [p2(a62, a29, X0), p6(X0, a19)]
3 :: 2 - 00:00:00 (4463 nps)

Query 92: [p9(X8, X8)]
2 :: 1 - 00:00:00 (39746 nps)

Query 93: [p6(X0, X1), p9(X0, X0), p5(a90, a40, X0), p6(X1, X2), p2(X2, a70, a89)]

98 :: 6 - 00:00:00 (11176 nps)

Query 94: [p1(X8, a0, X4, a13)]
3 :: 2 - 00:00:00 (5730 nps)

Query 95: [p1(X0, a0, a5, X0), p8(X0, a45)]
7 :: 4 - 00:00:00 (7131 nps)

Query 96: [p1(a93, a0, X0, X3)]
3 :: 2 - 00:00:00 (4660 nps)

Query 97: [p8(X7, X7)]
3 :: 2 - 00:00:00 (4696 nps)

Query 98: [p9(X2, X2)]
2 :: 1 - 00:00:00 (8684 nps)

Query 99: [p1(a17, X8, X2, a17)]
4 :: 3 - 00:00:00 (15145 nps)

Query 100: [p7(X0, a86), p5(X0, a22, a16)]
3 :: 2 - 00:00:00 (9787 nps)

std: 222.71 nodes/query

Time to run all queries: 0.8640817720000031

using cuda device

UNITY: ming

Embedding Model: rKB_model.pth

Scoring Model: uni_mr_model.pt

Control: mingoal

Rule Eval: Max Rule

Learned Embedding: Network using cuda

/opt/tljh/user/envs/cse327python311/lib/python3.11/site-packages/torch/nn/modules/lazy.py:180: UserWarning: Lazy modules are a new feature under heavy development so changes to the API or functionality can happen at any moment.

warnings.warn('Lazy modules are a new feature under heavy development ')

Query 1: [p2(a12, a72, X0), p0(X0, X1), p3(X1), p8(a63, X1)]
20 :: 15 - 00:00:00 (20 nps)

Query 2: [p1(X1, a0, a5, X0)]
3 :: 2 - 00:00:00 (27 nps)

Query 3: [p0(X1, X0), p6(a37, X0), p8(a77, X1)]
5 :: 3 - 00:00:00 (31 nps)

Query 4: [p1(X7, a0, a5, X7)]
4 :: 3 - 00:00:00 (44 nps)

Query 5: [p1(X1, a0, a5, X1)]
4 :: 3 - 00:00:00 (44 nps)

Query 6: [p1(X2, a0, a5, X6)]
3 :: 2 - 00:00:00 (28 nps)

Query 7: [p6(X0, a18)]

4 :: 3 - 00:00:00 (227 nps)

Query 8: [p2(X0, X1, a92), p6(a94, X0), p5(a0, X1, a39)]
4 :: 3 - 00:00:00 (107 nps)

Query 9: [p8(X3, X3)]
3 :: 2 - 00:00:00 (41 nps)

Query 10: [p5(a20, X0, a24), p1(a0, X0, a14, a10)]
3 :: 2 - 00:00:00 (292 nps)

Query 11: [p1(X6, X1, X7, X6)]
2 :: 1 - 00:00:00 (182 nps)

Query 12: [p9(X5, X5)]
3 :: 2 - 00:00:00 (36 nps)

Query 13: [p8(X1, X1)]
3 :: 2 - 00:00:00 (33 nps)

Query 14: [p4(X0, X1), p8(X0, X0), p2(a14, X1, X2), p8(X2, X2), p0(a0, X2)]
8 :: 7 - 00:00:00 (27 nps)

Query 15: [p8(X0, X0), p1(X0, a0, a5, X0)]
4 :: 3 - 00:00:00 (47 nps)

Query 16: [p9(a27, a27)]
3 :: 2 - 00:00:00 (384 nps)

Query 17: [p3(X0), p1(a41, a58, a10, X0), p9(X0, X0), p2(a75, a54, X0)]
87 :: 15 - 00:00:00 (98 nps)

Query 18: [p1(a1, X6, a5, X6)]
3 :: 2 - 00:00:00 (174 nps)

Query 19: [p1(a17, X1, X3, a17)]
4 :: 3 - 00:00:00 (185 nps)

Query 20: [p5(X0, a10, X1), p2(a40, X0, a11), p9(X1, X1)]
13 :: 4 - 00:00:00 (113 nps)

Query 21: [p1(X6, a0, X5, X6)]
4 :: 3 - 00:00:00 (41 nps)

Query 22: [p3(X7)]
2 :: 1 - 00:00:00 (32 nps)

Query 23: [p2(a98, X1, X0), p4(X0, a70), p7(X1, a35)]
4 :: 3 - 00:00:00 (68 nps)

Query 24: [p9(a27, a27)]
3 :: 2 - 00:00:00 (32198 nps)

Query 25: [p1(a11, a0, a5, a31)]
3 :: 2 - 00:00:00 (183 nps)

Query 26: [p8(a0, X0), p8(X0, X0)]

7 :: 3 - 00:00:00 (51 nps)

Query 27: [p9(a6, a6)]

3 :: 2 - 00:00:00 (474 nps)

Query 28: [p0(X0, a0), p2(X0, X2, X1), p7(a79, X1), p8(X2, X2)]

7566 :: 5 - 00:00:06 (1249 nps)

Query 29: [p9(X4, X4)]

3 :: 2 - 00:00:00 (34 nps)

Query 30: [p9(a38, X0), p1(a91, X0, a5, a91), p1(a25, X0, a23, a54)]

6 :: 5 - 00:00:00 (93 nps)

Query 31: [p1(X1, a0, X0, X1)]

4 :: 3 - 00:00:00 (658 nps)

Query 32: [p1(X0, X3, a5, X9)]

3 :: 2 - 00:00:00 (29 nps)

Query 33: [p1(X8, X8, a5, a77)]

3 :: 2 - 00:00:00 (182 nps)

Query 34: [p6(X9, X9)]

4 :: 3 - 00:00:00 (243 nps)

Query 35: [p5(a58, a1, X0), p8(X0, X0), p8(a93, X0)]

5 :: 4 - 00:00:00 (128 nps)

Query 36: [p1(a1, X9, X1, X9)]

3 :: 2 - 00:00:00 (772 nps)

Query 37: [p1(a78, a0, X0, a19), p1(a12, a0, X0, a53)]

5 :: 4 - 00:00:00 (92 nps)

Query 38: [p1(a6, a0, X2, X9)]

3 :: 2 - 00:00:00 (137 nps)

Query 39: [p3(X4)]

2 :: 1 - 00:00:00 (38 nps)

Query 40: [p8(X3, X3)]

3 :: 2 - 00:00:00 (27955 nps)

Query 41: [p5(a20, a74, X0), p1(X0, a43, a11, a50)]

3 :: 2 - 00:00:00 (509 nps)

Query 42: [p8(a19, X0), p1(X0, a0, X1, a0), p1(a19, a0, X1, X0), p4(X1, a48)]

6 :: 5 - 00:00:00 (60 nps)

Query 43: [p1(a71, X3, a5, X2)]

3 :: 2 - 00:00:00 (105 nps)

Query 44: [p0(X1, X0), p3(X1)]

112 :: 2 - 00:00:00 (179 nps)

Query 45: [p9(X4, X4)]
3 :: 2 - 00:00:00 (21677 nps)

Query 46: [p6(X5, X0)]
2 :: 1 - 00:00:00 (16 nps)

Query 47: [p9(X9, X9)]
3 :: 2 - 00:00:00 (34 nps)

Query 48: [p8(X2, X0), p7(X0, X1), p6(a33, X1), p5(a27, X2, a66)]
874 :: 4 - 00:00:05 (172 nps)

Query 49: [p1(a63, X1, a5, X0), p5(a13, a11, X0), p8(a1, X1)]
5 :: 4 - 00:00:00 (84 nps)

Query 50: [p1(a93, X1, X2, X5)]
3 :: 2 - 00:00:00 (163 nps)

Query 51: [p9(X6, X6)]
3 :: 2 - 00:00:00 (39 nps)

Query 52: [p6(a0, X0), p9(X0, a0), p1(X0, a0, a5, a13)]
6 :: 5 - 00:00:00 (52 nps)

Query 53: [p9(X8, X8)]
3 :: 2 - 00:00:00 (45 nps)

Query 54: [p0(X1, X0), p9(X0, X0), p5(X2, a0, X1), p5(a1, X3, X2), p4(a17, X3)]
7 :: 6 - 00:00:00 (22 nps)

Query 55: [p7(X0, X1)]
2 :: 1 - 00:00:00 (21292 nps)

Query 56: [p7(a22, X0), p2(X0, a70, a15)]
3 :: 2 - 00:00:00 (116 nps)

Query 57: [p9(X9, X9)]
3 :: 2 - 00:00:00 (27540 nps)

Query 58: [p1(a61, a0, X3, X7)]
3 :: 2 - 00:00:00 (200 nps)

Query 59: [p6(X7, a20)]
4 :: 3 - 00:00:00 (299 nps)

Query 60: [p0(X0, X1), p7(X0, X2), p8(X1, a19), p2(a19, X3, X2), p2(a57, X3, a15)]
6 :: 5 - 00:00:00 (15 nps)

Query 61: [p1(X0, a0, X4, X0)]
4 :: 3 - 00:00:00 (630 nps)

Query 62: [p1(X3, a0, X5, X3)]
4 :: 3 - 00:00:00 (41 nps)

Query 63: [p1(X3, a0, X9, X3)]

4 :: 3 - 00:00:00 (629 nps)

Query 64: [p1(X6, X5, a5, X6)]

4 :: 3 - 00:00:00 (590 nps)

Query 65: [p6(X0, a17), p2(X1, X0, a35), p7(X0, a0), p0(X1, a0), p8(X0, X0)]

7 :: 6 - 00:00:00 (21 nps)

Query 66: [p8(X8, X8)]

3 :: 2 - 00:00:00 (33 nps)

Query 67: [p1(a71, a0, X7, X1)]

3 :: 2 - 00:00:00 (108 nps)

Query 68: [p2(X0, a1, a49), p1(X0, a0, a5, X0)]

5 :: 4 - 00:00:00 (261 nps)

Query 69: [p2(X0, a54, a0), p5(X0, a22, a16), p4(a22, X0)]

4 :: 3 - 00:00:00 (178 nps)

Query 70: [p6(X1, X0), p4(X0, a75), p9(X0, X0), p5(a41, X1, a0)]

6 :: 5 - 00:00:00 (33 nps)

Query 71: [p0(X0, a4), p5(X0, a11, a50)]

3 :: 2 - 00:00:00 (293 nps)

Query 72: [p8(X9, X9)]

3 :: 2 - 00:00:00 (33 nps)

Query 73: [p0(X0, a88), p5(X0, a40, a27)]

3 :: 2 - 00:00:00 (297 nps)

Query 74: [p1(X0, X1, a5, X0), p2(X1, a14, X0), p1(a77, X1, a5, a10), p9(a91, X1)]

8 :: 7 - 00:00:00 (78 nps)

Query 75: [p1(a22, X2, a5, a11)]

3 :: 2 - 00:00:00 (223 nps)

Query 76: [p1(a71, a0, a5, X7)]

3 :: 2 - 00:00:00 (162 nps)

Query 77: [p1(a0, a0, a5, X6)]

3 :: 2 - 00:00:00 (122 nps)

Query 78: [p0(X0, X1), p9(X0, X0), p3(X0), p6(X0, a1), p2(X1, a87, a11)]

7 :: 6 - 00:00:00 (44 nps)

Query 79: [p1(X0, X1, a5, X0), p2(a44, a20, X0), p8(X1, X1)]

7 :: 6 - 00:00:00 (114 nps)

Query 80: [p6(a0, X0)]

2 :: 1 - 00:00:00 (24067 nps)

Query 81: [p6(a0, a18)]

4 :: 3 - 00:00:00 (373 nps)

Query 82: [p6(X0, a16), p2(a48, X0, a34)]

3 :: 2 - 00:00:00 (124 nps)

Query 83: [p1(X0, X5, X3, X1)]

2 :: 1 - 00:00:00 (23 nps)

Query 84: [p9(X3, X3)]

3 :: 2 - 00:00:00 (35 nps)

Query 85: [p8(X5, X5)]

3 :: 2 - 00:00:00 (33 nps)

Query 86: [p1(X0, X4, X6, X0)]

4 :: 3 - 00:00:00 (360 nps)

Query 87: [p6(X8, X5)]

2 :: 1 - 00:00:00 (17 nps)

Query 88: [p1(a58, a0, X2, a11)]

3 :: 2 - 00:00:00 (182 nps)

Query 89: [p2(X0, a20, a10), p6(X0, a0)]

3 :: 2 - 00:00:00 (89 nps)

Query 90: [p5(a87, X0, a30), p7(X0, a94)]

3 :: 2 - 00:00:00 (199 nps)

Query 91: [p2(a62, a29, X0), p6(X0, a19)]

3 :: 2 - 00:00:00 (194 nps)

Query 92: [p9(X8, X8)]

3 :: 2 - 00:00:00 (27226 nps)

Query 93: [p6(X0, X1), p9(X0, X0), p5(a90, a40, X0), p6(X1, X2), p2(X2, a70, a89)]

7 :: 6 - 00:00:00 (20 nps)

Query 94: [p1(X8, a0, X4, a13)]

3 :: 2 - 00:00:00 (148 nps)

Query 95: [p1(X0, a0, a5, X0), p8(X0, a45)]

526 :: 4 - 00:00:00 (859 nps)

Query 96: [p1(a93, a0, X0, X3)]

3 :: 2 - 00:00:00 (169 nps)

Query 97: [p8(X7, X7)]

3 :: 2 - 00:00:00 (47 nps)

Query 98: [p9(X2, X2)]

3 :: 2 - 00:00:00 (40 nps)

Query 99: [p1(a17, X8, X2, a17)]

4 :: 3 - 00:00:00 (606 nps)

Query 100: [p7(X0, a86), p5(X0, a22, a16)]

3 :: 2 - 00:00:00 (191 nps)

ming: 95.47 nodes/query
Time to run all queries: 20.200299929999999

STEP 7: By default, the system uses an embedding size of 50, meaning that the meaning of every atom is represented by a 50-dimensional vector of numbers.

Select a different embedding size and train a new embedding model and scoring model for your custom KB as well as each of your random KBs. To specify an embedding size, you can add the `--embed_size` argument when running `kbencoder` with the `train_unification_model` option. The same option (with the same value) should be used when running `nnreasoner` and.

To avoid overwriting your `rKB_model.pth` file, use `--embed_model_path` to specify and use an alternate name when running `kbencoder`, `nnreasoner`, and `evaluate`. To avoid overwriting your `uni_mr_model.pt` file, use `--save_model` with `nnreasoner` and `--scoring_model_path` with `evaluate`.

You can (and should) use the same `vocab.pkl`, `mr_train_examples.csv`, and `test_queries.txt` that you used in the previous steps. Assuming you have run the code for each KB in a separate directory, you do not need to rerun any other steps than the two training steps and the evaluation.

Does your new embedding size lead to a better or worse training loss for the embedding and scoring model? Does it improve or worsen performance on the test queries? Why do you think this is?

```
In [13]: !python ../kbencoder.py --kb_path randomKB.txt --train_unification_model --embed_si
```

Using cuda device

Uni model embed size: 25

```
[████████████████████] 3500/3500 Anchors generated  
[████████████████████] 70000/70000 Triplets generated  
[██████-----] 18403/70000 Triplet encodings
```

IOPub data rate exceeded.

The Jupyter server will temporarily stop sending output to the client in order to avoid crashing it.

To change this limit, set the config variable

`--ServerApp.iopub_data_rate_limit`.`

Current values:

`ServerApp.iopub_data_rate_limit=1000000.0 (bytes/sec)`

`ServerApp.rate_limit_window=3.0 (secs)`

```
[██████████-----] 34971/70000 Triplet encodings
```

IOPub data rate exceeded.
The Jupyter server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--ServerApp.iopub\_data\_rate\_limit`.

Current values:
ServerApp.iopub_data_rate_limit=1000000.0 (bytes/sec)
ServerApp.rate_limit_window=3.0 (secs)

[██████████-----] 43267/70000 Triplet encodings

IOPub data rate exceeded.
The Jupyter server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--ServerApp.iopub\_data\_rate\_limit`.

Current values:
ServerApp.iopub_data_rate_limit=1000000.0 (bytes/sec)
ServerApp.rate_limit_window=3.0 (secs)

[██████████---] 61667/70000 Triplet encodings

IOPub data rate exceeded.
The Jupyter server will temporarily stop sending output
to the client in order to avoid crashing it.
To change this limit, set the config variable
`--ServerApp.iopub\_data\_rate\_limit`.

Current values:
ServerApp.iopub_data_rate_limit=1000000.0 (bytes/sec)
ServerApp.rate_limit_window=3.0 (secs)

[██████████████████] 70000/70000 Triplet encodings

Early stopping triggered-----] 20.4% Epoch | [Epoch 103/500, Training Loss:
0.2481, Validation Loss: 0.4884]
Saved model.
Saved plot.
unification model generated

In [14]: !python ../nnreasoner.py --embed_type unification --save_model mr_model_size_25.pt

States from vocab: vocab
Num predicates: 10
Max arity: 4
Num constants: 100
Num variables: 10

Training unification

Embed size: 25

Reading examples from mr_train_examples.csv

Parsing examples...

Generating embeddings...

Embedding input size: 450

[████████████████] 2947/2947 Prepping examples

Loaded data...

Using cuda device

/opt/tljh/user/envs/cse327python311/lib/python3.11/site-packages/torch/nn/modules/lazy.py:180: UserWarning: Lazy modules are a new feature under heavy development so changes to the API or functionality can happen at any moment.

warnings.warn('Lazy modules are a new feature under heavy development '

0	0.58275 (-)
50	0.25503 (-0.0068)
100	0.22526 (-0.0036)
150	0.20657 (-0.0025)
200	0.18875 (-0.0020)
250	0.18031 (-0.0016)
300	0.17025 (-0.0014)
350	0.16453 (-0.0012)
400	0.15660 (-0.0011)
450	0.15200 (-0.0009)
500	0.14575 (-0.0009)
550	0.14117 (-0.0008)
600	0.13682 (-0.0007)
650	0.13452 (-0.0007)
700	0.13136 (-0.0006)
750	0.12681 (-0.0006)
800	0.12379 (-0.0005)
850	0.12148 (-0.0005)
900	0.11766 (-0.0005)
950	0.11640 (-0.0005)
1000	0.11366 (-0.0004)
1050	0.11066 (-0.0004)
1100	0.10767 (-0.0004)
1150	0.10586 (-0.0004)
1200	0.10173 (-0.0004)
1250	0.10224 (-0.0003)
1300	0.10034 (-0.0003)
1350	0.09746 (-0.0003)
1400	0.09530 (-0.0003)
1450	0.09321 (-0.0003)
1500	0.09380 (-0.0003)
1550	0.09284 (-0.0003)
1600	0.09093 (-0.0003)
1650	0.08773 (-0.0003)
1700	0.08690 (-0.0003)
1750	0.08436 (-0.0002)
1800	0.08698 (-0.0002)

```
1850    0.08054 (-0.0002)
1900    0.08031 (-0.0002)
1950    0.08043 (-0.0002)
2000    0.07801 (-0.0002)
2050    0.08014 (-0.0002)
2100    0.07600 (-0.0002)
2150    0.07460 (-0.0002)
2200    0.07345 (-0.0002)
2250    0.07099 (-0.0002)
2300    0.07005 (-0.0002)
2350    0.07188 (-0.0002)
2400    0.07194 (-0.0002)
2450    0.06863 (-0.0002)
2500    0.06820 (-0.0002)
2550    0.06754 (-0.0002)
2600    0.06387 (-0.0002)
2650    0.06515 (-0.0002)
2700    0.06402 (-0.0001)
2750    0.06349 (-0.0001)
2800    0.06325 (-0.0001)
2850    0.06274 (-0.0001)
2900    0.06219 (-0.0001)
2950    0.06176 (-0.0001)
2999    0.06102 (-0.0001)
Saved training loss figure.
```

```
In [17]: !python ../evaluate.py --kb randomKB.txt -s --embed_size 25 --embed_model_path rKB_
```

using cuda device
STANDARD

Query 1: [p2(a12, a72, X0), p0(X0, X1), p3(X1), p8(a63, X1)]
5 :: 4 - 00:00:00 (3314 nps)

Query 2: [p1(X1, a0, a5, X0)]
3 :: 2 - 00:00:00 (3669 nps)

Query 3: [p0(X1, X0), p6(a37, X0), p8(a77, X1)]
78 :: 3 - 00:00:00 (6882 nps)

Query 4: [p1(X7, a0, a5, X7)]
4 :: 3 - 00:00:00 (3910 nps)

Query 5: [p1(X1, a0, a5, X1)]
4 :: 3 - 00:00:00 (3946 nps)

Query 6: [p1(X2, a0, a5, X6)]
3 :: 2 - 00:00:00 (3899 nps)

Query 7: [p6(X0, a18)]
4 :: 3 - 00:00:00 (3123 nps)

Query 8: [p2(X0, X1, a92), p6(a94, X0), p5(a0, X1, a39)]
4 :: 3 - 00:00:00 (4064 nps)

Query 9: [p8(X3, X3)]
3 :: 2 - 00:00:00 (4198 nps)

Query 10: [p5(a20, X0, a24), p1(a0, X0, a14, a10)]
3 :: 2 - 00:00:00 (5445 nps)

Query 11: [p1(X6, X1, X7, X6)]
2 :: 1 - 00:00:00 (4911 nps)

Query 12: [p9(X5, X5)]
2 :: 1 - 00:00:00 (7708 nps)

Query 13: [p8(X1, X1)]
3 :: 2 - 00:00:00 (4502 nps)

Query 14: [p4(X0, X1), p8(X0, X0), p2(a14, X1, X2), p8(X2, X2), p0(a0, X2)]
102 :: 7 - 00:00:00 (5040 nps)

Query 15: [p8(X0, X0), p1(X0, a0, a5, X0)]
6 :: 5 - 00:00:00 (2744 nps)

Query 16: [p9(a27, a27)]
3 :: 2 - 00:00:00 (7128 nps)

Query 17: [p3(X0), p1(a41, a58, a10, X0), p9(X0, X0), p2(a75, a54, X0)]
58 :: 11 - 00:00:00 (4506 nps)

Query 18: [p1(a1, X6, a5, X6)]
3 :: 2 - 00:00:00 (4412 nps)

Query 19: [p1(a17, X1, X3, a17)]

4 :: 3 - 00:00:00 (4463 nps)

Query 20: [p5(X0, a10, X1), p2(a40, X0, a11), p9(X1, X1)]

5 :: 4 - 00:00:00 (3435 nps)

Query 21: [p1(X6, a0, X5, X6)]

4 :: 3 - 00:00:00 (3450 nps)

Query 22: [p3(X7)]

2 :: 1 - 00:00:00 (6631 nps)

Query 23: [p2(a98, X1, X0), p4(X0, a70), p7(X1, a35)]

4 :: 3 - 00:00:00 (3754 nps)

Query 24: [p9(a27, a27)]

3 :: 2 - 00:00:00 (46642 nps)

Query 25: [p1(a11, a0, a5, a31)]

3 :: 2 - 00:00:00 (5735 nps)

Query 26: [p8(a0, X0), p8(X0, X0)]

13 :: 3 - 00:00:00 (4842 nps)

Query 27: [p9(a6, a6)]

3 :: 2 - 00:00:00 (6025 nps)

Query 28: [p0(X0, a0), p2(X0, X2, X1), p7(a79, X1), p8(X2, X2)]

8 :: 5 - 00:00:00 (3386 nps)

Query 29: [p9(X4, X4)]

2 :: 1 - 00:00:00 (6140 nps)

Query 30: [p9(a38, X0), p1(a91, X0, a5, a91), p1(a25, X0, a23, a54)]

6 :: 5 - 00:00:00 (3315 nps)

Query 31: [p1(X1, a0, X0, X1)]

4 :: 3 - 00:00:00 (10070 nps)

Query 32: [p1(X0, X3, a5, X9)]

3 :: 2 - 00:00:00 (3956 nps)

Query 33: [p1(X8, X8, a5, a77)]

3 :: 2 - 00:00:00 (5309 nps)

Query 34: [p6(X9, X9)]

4 :: 3 - 00:00:00 (5326 nps)

Query 35: [p5(a58, a1, X0), p8(X0, X0), p8(a93, X0)]

5 :: 4 - 00:00:00 (6161 nps)

Query 36: [p1(a1, X9, X1, X9)]

3 :: 2 - 00:00:00 (7970 nps)

Query 37: [p1(a78, a0, X0, a19), p1(a12, a0, X0, a53)]

5 :: 4 - 00:00:00 (4745 nps)

Query 38: [p1(a6, a0, X2, X9)]

3 :: 2 - 00:00:00 (3767 nps)

Query 39: [p3(X4)]

2 :: 1 - 00:00:00 (6856 nps)

Query 40: [p8(X3, X3)]

3 :: 2 - 00:00:00 (43921 nps)

Query 41: [p5(a20, a74, X0), p1(X0, a43, a11, a50)]

3 :: 2 - 00:00:00 (4612 nps)

Query 42: [p8(a19, X0), p1(X0, a0, X1, a0), p1(a19, a0, X1, X0), p4(X1, a48)]

7 :: 6 - 00:00:00 (4282 nps)

Query 43: [p1(a71, X3, a5, X2)]

3 :: 2 - 00:00:00 (4234 nps)

Query 44: [p0(X1, X0), p3(X1)]

212 :: 2 - 00:00:00 (1122 nps)

Query 45: [p9(X4, X4)]

2 :: 1 - 00:00:00 (29366 nps)

Query 46: [p6(X5, X0)]

2 :: 1 - 00:00:00 (3764 nps)

Query 47: [p9(X9, X9)]

2 :: 1 - 00:00:00 (7878 nps)

Query 48: [p8(X2, X0), p7(X0, X1), p6(a33, X1), p5(a27, X2, a66)]

1040 :: 4 - 00:00:00 (7662 nps)

Query 49: [p1(a63, X1, a5, X0), p5(a13, a11, X0), p8(a1, X1)]

5 :: 4 - 00:00:00 (4513 nps)

Query 50: [p1(a93, X1, X2, X5)]

3 :: 2 - 00:00:00 (4369 nps)

Query 51: [p9(X6, X6)]

2 :: 1 - 00:00:00 (8801 nps)

Query 52: [p6(a0, X0), p9(X0, a0), p1(X0, a0, a5, a13)]

9 :: 6 - 00:00:00 (5704 nps)

Query 53: [p9(X8, X8)]

2 :: 1 - 00:00:00 (8713 nps)

Query 54: [p0(X1, X0), p9(X0, X0), p5(X2, a0, X1), p5(a1, X3, X2), p4(a17, X3)]

69 :: 6 - 00:00:00 (7520 nps)

Query 55: [p7(X0, X1)]

2 :: 1 - 00:00:00 (4686 nps)

Query 56: [p7(a22, X0), p2(X0, a70, a15)]

3 :: 2 - 00:00:00 (4819 nps)

Query 57: [p9(X9, X9)]

2 :: 1 - 00:00:00 (33589 nps)

Query 58: [p1(a61, a0, X3, X7)]

3 :: 2 - 00:00:00 (4575 nps)

Query 59: [p6(X7, a20)]

2 :: 1 - 00:00:00 (6283 nps)

Query 60: [p0(X0, X1), p7(X0, X2), p8(X1, a19), p2(a19, X3, X2), p2(a57, X3, a15)]

1192 :: 5 - 00:00:00 (8841 nps)

Query 61: [p1(X0, a0, X4, X0)]

4 :: 3 - 00:00:00 (4209 nps)

Query 62: [p1(X3, a0, X5, X3)]

4 :: 3 - 00:00:00 (4315 nps)

Query 63: [p1(X3, a0, X9, X3)]

4 :: 3 - 00:00:00 (11511 nps)

Query 64: [p1(X6, X5, a5, X6)]

4 :: 3 - 00:00:00 (12545 nps)

Query 65: [p6(X0, a17), p2(X1, X0, a35), p7(X0, a0), p0(X1, a0), p8(X0, X0)]

7 :: 6 - 00:00:00 (4392 nps)

Query 66: [p8(X8, X8)]

3 :: 2 - 00:00:00 (4771 nps)

Query 67: [p1(a71, a0, X7, X1)]

3 :: 2 - 00:00:00 (4759 nps)

Query 68: [p2(X0, a1, a49), p1(X0, a0, a5, X0)]

5 :: 4 - 00:00:00 (5179 nps)

Query 69: [p2(X0, a54, a0), p5(X0, a22, a16), p4(a22, X0)]

4 :: 3 - 00:00:00 (5367 nps)

Query 70: [p6(X1, X0), p4(X0, a75), p9(X0, X0), p5(a41, X1, a0)]

27 :: 5 - 00:00:00 (7352 nps)

Query 71: [p0(X0, a4), p5(X0, a11, a50)]

3 :: 2 - 00:00:00 (5461 nps)

Query 72: [p8(X9, X9)]

3 :: 2 - 00:00:00 (4699 nps)

Query 73: [p0(X0, a88), p5(X0, a40, a27)]

3 :: 2 - 00:00:00 (5543 nps)

Query 74: [p1(X0, X1, a5, X0), p2(X1, a14, X0), p1(a77, X1, a5, a10), p9(a91, X1)]

33 :: 7 - 00:00:00 (5002 nps)

Query 75: [p1(a22, X2, a5, a11)]
3 :: 2 - 00:00:00 (6236 nps)

Query 76: [p1(a71, a0, a5, X7)]
3 :: 2 - 00:00:00 (5134 nps)

Query 77: [p1(a0, a0, a5, X6)]
3 :: 2 - 00:00:00 (5137 nps)

Query 78: [p0(X0, X1), p9(X0, X0), p3(X0), p6(X0, a1), p2(X1, a87, a11)]
19040 :: 6 - 00:00:00 (58808 nps)

Query 79: [p1(X0, X1, a5, X0), p2(a44, a20, X0), p8(X1, X1)]
10 :: 6 - 00:00:00 (5982 nps)

Query 80: [p6(a0, X0)]
2 :: 1 - 00:00:00 (35592 nps)

Query 81: [p6(a0, a18)]
4 :: 3 - 00:00:00 (11588 nps)

Query 82: [p6(X0, a16), p2(a48, X0, a34)]
4 :: 2 - 00:00:00 (4806 nps)

Query 83: [p1(X0, X5, X3, X1)]
2 :: 1 - 00:00:00 (4797 nps)

Query 84: [p9(X3, X3)]
2 :: 1 - 00:00:00 (8479 nps)

Query 85: [p8(X5, X5)]
3 :: 2 - 00:00:00 (4596 nps)

Query 86: [p1(X0, X4, X6, X0)]
2 :: 1 - 00:00:00 (5362 nps)

Query 87: [p6(X8, X5)]
2 :: 1 - 00:00:00 (4261 nps)

Query 88: [p1(a58, a0, X2, a11)]
3 :: 2 - 00:00:00 (6332 nps)

Query 89: [p2(X0, a20, a10), p6(X0, a0)]
3 :: 2 - 00:00:00 (4867 nps)

Query 90: [p5(a87, X0, a30), p7(X0, a94)]
3 :: 2 - 00:00:00 (5397 nps)

Query 91: [p2(a62, a29, X0), p6(X0, a19)]
3 :: 2 - 00:00:00 (4473 nps)

Query 92: [p9(X8, X8)]
2 :: 1 - 00:00:00 (39970 nps)

Query 93: [p6(X0, X1), p9(X0, X0), p5(a90, a40, X0), p6(X1, X2), p2(X2, a70, a89)]

98 :: 6 - 00:00:00 (11022 nps)

Query 94: [p1(X8, a0, X4, a13)]
3 :: 2 - 00:00:00 (5653 nps)

Query 95: [p1(X0, a0, a5, X0), p8(X0, a45)]
7 :: 4 - 00:00:00 (7186 nps)

Query 96: [p1(a93, a0, X0, X3)]
3 :: 2 - 00:00:00 (4651 nps)

Query 97: [p8(X7, X7)]
3 :: 2 - 00:00:00 (4692 nps)

Query 98: [p9(X2, X2)]
2 :: 1 - 00:00:00 (8615 nps)

Query 99: [p1(a17, X8, X2, a17)]
4 :: 3 - 00:00:00 (15518 nps)

Query 100: [p7(X0, a86), p5(X0, a22, a16)]
3 :: 2 - 00:00:00 (9399 nps)

std: 222.71 nodes/query

Time to run all queries: 0.9194828180000023

In [18]: !python ../evaluate.py --kb randomKB.txt -u --embed_size 25 --embed_model_path rKB_

```

using cuda device
UNITY: ming
    Embedding Model: rKB_model_e_size_25.pth
    Scoring Model: mr_model_size_25.pt

    Control: mingoal
    Rule Eval: Max Rule
Learned Embedding: Network using cuda
/opt/tljh/user/envs/cse327python311/lib/python3.11/site-packages/torch/nn/modules/la
zy.py:180: UserWarning: Lazy modules are a new feature under heavy development so ch
anges to the API or functionality can happen at any moment.
  warnings.warn('Lazy modules are a new feature under heavy development '
Query 1: [p2(a12, a72, X0), p0(X0, X1), p3(X1), p8(a63, X1)]
20 :: 15 - 00:00:01 (14 nps)

Query 2: [p1(X1, a0, a5, X0)]
3 :: 2 - 00:00:00 (15 nps)

Query 3: [p0(X1, X0), p6(a37, X0), p8(a77, X1)]
5 :: 3 - 00:00:00 (17 nps)

Query 4: [p1(X7, a0, a5, X7)]
4 :: 3 - 00:00:00 (22 nps)

Query 5: [p1(X1, a0, a5, X1)]
4 :: 3 - 00:00:00 (25 nps)

Query 6: [p1(X2, a0, a5, X6)]
3 :: 2 - 00:00:00 (16 nps)

Query 7: [p6(X0, a18)]
4 :: 3 - 00:00:00 (143 nps)

Query 8: [p2(X0, X1, a92), p6(a94, X0), p5(a0, X1, a39)]
4 :: 3 - 00:00:00 (79 nps)

Query 9: [p8(X3, X3)]
3 :: 2 - 00:00:00 (20 nps)

Query 10: [p5(a20, X0, a24), p1(a0, X0, a14, a10)]
3 :: 2 - 00:00:00 (138 nps)

Query 11: [p1(X6, X1, X7, X6)]
2 :: 1 - 00:00:00 (117 nps)

Query 12: [p9(X5, X5)]
2 :: 1 - 00:00:00 (172 nps)

Query 13: [p8(X1, X1)]
3 :: 2 - 00:00:00 (20 nps)

Query 14: [p4(X0, X1), p8(X0, X0), p2(a14, X1, X2), p8(X2, X2), p0(a0, X2)]
11 :: 7 - 00:00:01 (9 nps)

Query 15: [p8(X0, X0), p1(X0, a0, a5, X0)]
6 :: 5 - 00:00:00 (10 nps)

```

Query 16: [p9(a27, a27)]
3 :: 2 - 00:00:00 (240 nps)

Query 17: [p3(X0), p1(a41, a58, a10, X0), p9(X0, X0), p2(a75, a54, X0)]
99 :: 15 - 00:00:01 (79 nps)

Query 18: [p1(a1, X6, a5, X6)]
3 :: 2 - 00:00:00 (114 nps)

Query 19: [p1(a17, X1, X3, a17)]
4 :: 3 - 00:00:00 (118 nps)

Query 20: [p5(X0, a10, X1), p2(a40, X0, a11), p9(X1, X1)]
5 :: 4 - 00:00:00 (90 nps)

Query 21: [p1(X6, a0, X5, X6)]
4 :: 3 - 00:00:00 (24 nps)

Query 22: [p3(X7)]
2 :: 1 - 00:00:00 (20 nps)

Query 23: [p2(a98, X1, X0), p4(X0, a70), p7(X1, a35)]
4 :: 3 - 00:00:00 (50 nps)

Query 24: [p9(a27, a27)]
3 :: 2 - 00:00:00 (31221 nps)

Query 25: [p1(a11, a0, a5, a31)]
3 :: 2 - 00:00:00 (112 nps)

Query 26: [p8(a0, X0), p8(X0, X0)]
155 :: 3 - 00:00:01 (119 nps)

Query 27: [p9(a6, a6)]
3 :: 2 - 00:00:00 (241 nps)

Query 28: [p0(X0, a0), p2(X0, X2, X1), p7(a79, X1), p8(X2, X2)]
1062 :: 5 - 00:00:10 (104 nps)

Query 29: [p9(X4, X4)]
2 :: 1 - 00:00:00 (180 nps)

Query 30: [p9(a38, X0), p1(a91, X0, a5, a91), p1(a25, X0, a23, a54)]
6 :: 5 - 00:00:00 (60 nps)

Query 31: [p1(X1, a0, X0, X1)]
4 :: 3 - 00:00:00 (400 nps)

Query 32: [p1(X0, X3, a5, X9)]
3 :: 2 - 00:00:00 (16 nps)

Query 33: [p1(X8, X8, a5, a77)]
3 :: 2 - 00:00:00 (113 nps)

Query 34: [p6(X9, X9)]

4 :: 3 - 00:00:00 (158 nps)

Query 35: [p5(a58, a1, X0), p8(X0, X0), p8(a93, X0)]
5 :: 4 - 00:00:00 (56 nps)

Query 36: [p1(a1, X9, X1, X9)]
3 :: 2 - 00:00:00 (265 nps)

Query 37: [p1(a78, a0, X0, a19), p1(a12, a0, X0, a53)]
5 :: 4 - 00:00:00 (65 nps)

Query 38: [p1(a6, a0, X2, X9)]
3 :: 2 - 00:00:00 (85 nps)

Query 39: [p3(X4)]
2 :: 1 - 00:00:00 (19 nps)

Query 40: [p8(X3, X3)]
3 :: 2 - 00:00:00 (27949 nps)

Query 41: [p5(a20, a74, X0), p1(X0, a43, a11, a50)]
3 :: 2 - 00:00:00 (140 nps)

Query 42: [p8(a19, X0), p1(X0, a0, X1, a0), p1(a19, a0, X1, X0), p4(X1, a48)]
6 :: 5 - 00:00:00 (26 nps)

Query 43: [p1(a71, X3, a5, X2)]
3 :: 2 - 00:00:00 (81 nps)

Query 44: [p0(X1, X0), p3(X1)]
4 :: 2 - 00:00:00 (973 nps)

Query 45: [p9(X4, X4)]
2 :: 1 - 00:00:00 (29049 nps)

Query 46: [p6(X5, X0)]
2 :: 1 - 00:00:00 (12 nps)

Query 47: [p9(X9, X9)]
2 :: 1 - 00:00:00 (204 nps)

Query 48: [p8(X2, X0), p7(X0, X1), p6(a33, X1), p5(a27, X2, a66)]
53109 :: 4 - 00:00:15 (3448 nps)

Query 49: [p1(a63, X1, a5, X0), p5(a13, a11, X0), p8(a1, X1)]
194 :: 4 - 00:00:00 (249 nps)

Query 50: [p1(a93, X1, X2, X5)]
3 :: 2 - 00:00:00 (76 nps)

Query 51: [p9(X6, X6)]
2 :: 1 - 00:00:00 (169 nps)

Query 52: [p6(a0, X0), p9(X0, a0), p1(X0, a0, a5, a13)]
75 :: 6 - 00:00:00 (96 nps)

Query 53: [p9(X8, X8)]
2 :: 1 - 00:00:00 (178 nps)

Query 54: [p0(X1, X0), p9(X0, X0), p5(X2, a0, X1), p5(a1, X3, X2), p4(a17, X3)]
7 :: 6 - 00:00:00 (22 nps)

Query 55: [p7(X0, X1)]
2 :: 1 - 00:00:00 (21494 nps)

Query 56: [p7(a22, X0), p2(X0, a70, a15)]
3 :: 2 - 00:00:00 (51 nps)

Query 57: [p9(X9, X9)]
2 :: 1 - 00:00:00 (25398 nps)

Query 58: [p1(a61, a0, X3, X7)]
3 :: 2 - 00:00:00 (88 nps)

Query 59: [p6(X7, a20)]
2 :: 1 - 00:00:00 (142 nps)

Query 60: [p0(X0, X1), p7(X0, X2), p8(X1, a19), p2(a19, X3, X2), p2(a57, X3, a15)]
6 :: 5 - 00:00:00 (6 nps)

Query 61: [p1(X0, a0, X4, X0)]
4 :: 3 - 00:00:00 (25 nps)

Query 62: [p1(X3, a0, X5, X3)]
4 :: 3 - 00:00:00 (25 nps)

Query 63: [p1(X3, a0, X9, X3)]
4 :: 3 - 00:00:00 (402 nps)

Query 64: [p1(X6, X5, a5, X6)]
4 :: 3 - 00:00:00 (25 nps)

Query 65: [p6(X0, a17), p2(X1, X0, a35), p7(X0, a0), p0(X1, a0), p8(X0, X0)]
7 :: 6 - 00:00:00 (28 nps)

Query 66: [p8(X8, X8)]
3 :: 2 - 00:00:00 (20 nps)

Query 67: [p1(a71, a0, X7, X1)]
3 :: 2 - 00:00:00 (67 nps)

Query 68: [p2(X0, a1, a49), p1(X0, a0, a5, X0)]
5 :: 4 - 00:00:00 (94 nps)

Query 69: [p2(X0, a54, a0), p5(X0, a22, a16), p4(a22, X0)]
4 :: 3 - 00:00:00 (114 nps)

Query 70: [p6(X1, X0), p4(X0, a75), p9(X0, X0), p5(a41, X1, a0)]
9 :: 5 - 00:00:00 (30 nps)

Query 71: [p0(X0, a4), p5(X0, a11, a50)]
3 :: 2 - 00:00:00 (177 nps)

Query 72: [p8(X9, X9)]

3 :: 2 - 00:00:00 (20 nps)

Query 73: [p0(X0, a88), p5(X0, a40, a27)]

3 :: 2 - 00:00:00 (202 nps)

Query 74: [p1(X0, X1, a5, X0), p2(X1, a14, X0), p1(a77, X1, a5, a10), p9(a91, X1)]

8 :: 7 - 00:00:00 (41 nps)

Query 75: [p1(a22, X2, a5, a11)]

3 :: 2 - 00:00:00 (117 nps)

Query 76: [p1(a71, a0, a5, X7)]

3 :: 2 - 00:00:00 (67 nps)

Query 77: [p1(a0, a0, a5, X6)]

3 :: 2 - 00:00:00 (49 nps)

Query 78: [p0(X0, X1), p9(X0, X0), p3(X0), p6(X0, a1), p2(X1, a87, a11)]

20 :: 15 - 00:00:00 (30 nps)

Query 79: [p1(X0, X1, a5, X0), p2(a44, a20, X0), p8(X1, X1)]

7 :: 6 - 00:00:00 (52 nps)

Query 80: [p6(a0, X0)]

2 :: 1 - 00:00:00 (21016 nps)

Query 81: [p6(a0, a18)]

4 :: 3 - 00:00:00 (192 nps)

Query 82: [p6(X0, a16), p2(a48, X0, a34)]

3 :: 2 - 00:00:00 (75 nps)

Query 83: [p1(X0, X5, X3, X1)]

2 :: 1 - 00:00:00 (13 nps)

Query 84: [p9(X3, X3)]

2 :: 1 - 00:00:00 (156 nps)

Query 85: [p8(X5, X5)]

3 :: 2 - 00:00:00 (20 nps)

Query 86: [p1(X0, X4, X6, X0)]

2 :: 1 - 00:00:00 (102 nps)

Query 87: [p6(X8, X5)]

2 :: 1 - 00:00:00 (9 nps)

Query 88: [p1(a58, a0, X2, a11)]

3 :: 2 - 00:00:00 (107 nps)

Query 89: [p2(X0, a20, a10), p6(X0, a0)]

7 :: 2 - 00:00:00 (142 nps)

Query 90: [p5(a87, X0, a30), p7(X0, a94)]

3 :: 2 - 00:00:00 (51 nps)

Query 91: [p2(a62, a29, X0), p6(X0, a19)]

3 :: 2 - 00:00:00 (85 nps)

Query 92: [p9(X8, X8)]

2 :: 1 - 00:00:00 (21347 nps)

Query 93: [p6(X0, X1), p9(X0, X0), p5(a90, a40, X0), p6(X1, X2), p2(X2, a70, a89)]

7 :: 6 - 00:00:00 (9 nps)

Query 94: [p1(X8, a0, X4, a13)]

3 :: 2 - 00:00:00 (88 nps)

Query 95: [p1(X0, a0, a5, X0), p8(X0, a45)]

275 :: 4 - 00:00:00 (454 nps)

Query 96: [p1(a93, a0, X0, X3)]

3 :: 2 - 00:00:00 (76 nps)

Query 97: [p8(X7, X7)]

3 :: 2 - 00:00:00 (20 nps)

Query 98: [p9(X2, X2)]

2 :: 1 - 00:00:00 (179 nps)

Query 99: [p1(a17, X8, X2, a17)]

4 :: 3 - 00:00:00 (390 nps)

Query 100: [p7(X0, a86), p5(X0, a22, a16)]

3 :: 2 - 00:00:00 (57 nps)

ming: 553.4 nodes/query

Time to run all queries: 42.270747549999999

STEP 8/9: Train and evaluate your modified model on all four KBs.

NOTE: I don't need to re-run kbencoder.py since step 8 only asks to modify the architecture of the scoring model (nnreasoner). As such, the embeddings (the meaning of the atoms) haven't changed, so the existing rKB_model.pth and vocab.pkl files are still valid.

```
In [26]: # Train modified unification model
!python ../nnreasoner-modified.py --embed_type unification --save_model mod_uni_mod
```

States from vocab: vocab
Num predicates: 10
Max arity: 4
Num constants: 100
Num variables: 10

Training unification

Embed size: 50

Reading examples from mr_train_examples.csv

Parsing examples...

Generating embeddings...

Embedding input size: 450

[████████████████████] 2947/2947 Prepping examples

Loaded data...

Using cuda device

/opt/tljh/user/envs/cse327python311/lib/python3.11/site-packages/torch/nn/modules/lazy.py:180: UserWarning: Lazy modules are a new feature under heavy development so changes to the API or functionality can happen at any moment.

warnings.warn('Lazy modules are a new feature under heavy development '

0	0.70841 (-)
50	0.56618 (-0.0026)
100	0.23475 (-0.0044)
150	0.19826 (-0.0033)
200	0.17982 (-0.0026)
250	0.17035 (-0.0021)
300	0.16111 (-0.0018)
350	0.15456 (-0.0015)
400	0.13948 (-0.0014)
450	0.13602 (-0.0012)
500	0.13285 (-0.0011)
550	0.12770 (-0.0010)
600	0.12097 (-0.0009)
650	0.11092 (-0.0009)
700	0.11180 (-0.0008)
750	0.10659 (-0.0008)
800	0.09802 (-0.0007)
850	0.09705 (-0.0007)
900	0.09027 (-0.0006)
950	0.08851 (-0.0006)
1000	0.09027 (-0.0006)
1050	0.08462 (-0.0005)
1100	0.08389 (-0.0005)
1150	0.08589 (-0.0005)
1200	0.08635 (-0.0005)
1250	0.08250 (-0.0005)
1300	0.08568 (-0.0004)
1350	0.07856 (-0.0004)
1400	0.07300 (-0.0004)
1450	0.07630 (-0.0004)
1500	0.07094 (-0.0004)
1550	0.07084 (-0.0004)
1600	0.07100 (-0.0004)
1650	0.06758 (-0.0003)
1700	0.07205 (-0.0003)
1750	0.06881 (-0.0003)
1800	0.06422 (-0.0003)

1850	0.06608	(-0.0003)
1900	0.06601	(-0.0003)
1950	0.05989	(-0.0003)
2000	0.06447	(-0.0003)
2050	0.06400	(-0.0003)
2100	0.06279	(-0.0003)
2150	0.05889	(-0.0003)
2200	0.06366	(-0.0002)
2250	0.05656	(-0.0002)
2300	0.05539	(-0.0002)
2350	0.05743	(-0.0002)
2400	0.05580	(-0.0002)
2450	0.05775	(-0.0002)
2500	0.05512	(-0.0002)
2550	0.05600	(-0.0002)
2600	0.06056	(-0.0002)
2650	0.05480	(-0.0002)
2700	0.05373	(-0.0002)
2750	0.05100	(-0.0002)
2800	0.05682	(-0.0002)
2850	0.05378	(-0.0002)
2900	0.04816	(-0.0002)
2950	0.04893	(-0.0002)
3000	0.04841	(-0.0002)
3050	0.04899	(-0.0002)
3100	0.04687	(-0.0002)
3150	0.04477	(-0.0002)
3200	0.04496	(-0.0002)
3250	0.04285	(-0.0002)
3300	0.04236	(-0.0002)
3350	0.04012	(-0.0001)
3400	0.04600	(-0.0001)
3450	0.04326	(-0.0001)
3500	0.04232	(-0.0001)
3550	0.04590	(-0.0001)
3600	0.04142	(-0.0001)
3650	0.03664	(-0.0001)
3700	0.04114	(-0.0001)
3750	0.04203	(-0.0001)
3800	0.04139	(-0.0001)
3850	0.04109	(-0.0001)
3900	0.03819	(-0.0001)
3950	0.03526	(-0.0001)
4000	0.03959	(-0.0001)
4050	0.05430	(-0.0001)
4100	0.04756	(-0.0001)
4150	0.03763	(-0.0001)
4200	0.03527	(-0.0001)
4250	0.03719	(-0.0001)
4300	0.04109	(-0.0001)
4350	0.04437	(-0.0001)
4400	0.04126	(-0.0001)
4449	0.03499	(-0.0001)

Saved training loss figure.

```
In [29]: # Evaluate standard modified model
!python ../evaluate_modified.py --kb randomKB.txt -s --scoring_model_path mod_std_m

# Evaluate unification modified model
!python ../evaluate_modified.py --kb randomKB.txt -u --scoring_model_path mod_uni_m
```

using cuda device
STANDARD

Query 1: [p2(a12, a72, X0), p0(X0, X1), p3(X1), p8(a63, X1)]
5 :: 4 - 00:00:00 (3838 nps)

Query 2: [p1(X1, a0, a5, X0)]
3 :: 2 - 00:00:00 (4221 nps)

Query 3: [p0(X1, X0), p6(a37, X0), p8(a77, X1)]
78 :: 3 - 00:00:00 (7941 nps)

Query 4: [p1(X7, a0, a5, X7)]
4 :: 3 - 00:00:00 (4186 nps)

Query 5: [p1(X1, a0, a5, X1)]
4 :: 3 - 00:00:00 (4481 nps)

Query 6: [p1(X2, a0, a5, X6)]
3 :: 2 - 00:00:00 (4386 nps)

Query 7: [p6(X0, a18)]
4 :: 3 - 00:00:00 (3689 nps)

Query 8: [p2(X0, X1, a92), p6(a94, X0), p5(a0, X1, a39)]
4 :: 3 - 00:00:00 (4708 nps)

Query 9: [p8(X3, X3)]
3 :: 2 - 00:00:00 (4578 nps)

Query 10: [p5(a20, X0, a24), p1(a0, X0, a14, a10)]
3 :: 2 - 00:00:00 (5905 nps)

Query 11: [p1(X6, X1, X7, X6)]
2 :: 1 - 00:00:00 (5489 nps)

Query 12: [p9(X5, X5)]
2 :: 1 - 00:00:00 (9186 nps)

Query 13: [p8(X1, X1)]
3 :: 2 - 00:00:00 (5069 nps)

Query 14: [p4(X0, X1), p8(X0, X0), p2(a14, X1, X2), p8(X2, X2), p0(a0, X2)]
102 :: 7 - 00:00:00 (5778 nps)

Query 15: [p8(X0, X0), p1(X0, a0, a5, X0)]
6 :: 5 - 00:00:00 (3585 nps)

Query 16: [p9(a27, a27)]
3 :: 2 - 00:00:00 (7620 nps)

Query 17: [p3(X0), p1(a41, a58, a10, X0), p9(X0, X0), p2(a75, a54, X0)]
58 :: 11 - 00:00:00 (5291 nps)

Query 18: [p1(a1, X6, a5, X6)]
3 :: 2 - 00:00:00 (5052 nps)

Query 19: [p1(a17, X1, X3, a17)]

4 :: 3 - 00:00:00 (5412 nps)

Query 20: [p5(X0, a10, X1), p2(a40, X0, a11), p9(X1, X1)]

5 :: 4 - 00:00:00 (4281 nps)

Query 21: [p1(X6, a0, X5, X6)]

4 :: 3 - 00:00:00 (4304 nps)

Query 22: [p3(X7)]

2 :: 1 - 00:00:00 (8160 nps)

Query 23: [p2(a98, X1, X0), p4(X0, a70), p7(X1, a35)]

4 :: 3 - 00:00:00 (4300 nps)

Query 24: [p9(a27, a27)]

3 :: 2 - 00:00:00 (54355 nps)

Query 25: [p1(a11, a0, a5, a31)]

3 :: 2 - 00:00:00 (6108 nps)

Query 26: [p8(a0, X0), p8(X0, X0)]

13 :: 3 - 00:00:00 (5785 nps)

Query 27: [p9(a6, a6)]

3 :: 2 - 00:00:00 (7803 nps)

Query 28: [p0(X0, a0), p2(X0, X2, X1), p7(a79, X1), p8(X2, X2)]

8 :: 5 - 00:00:00 (3234 nps)

Query 29: [p9(X4, X4)]

2 :: 1 - 00:00:00 (6277 nps)

Query 30: [p9(a38, X0), p1(a91, X0, a5, a91), p1(a25, X0, a23, a54)]

6 :: 5 - 00:00:00 (2461 nps)

Query 31: [p1(X1, a0, X0, X1)]

4 :: 3 - 00:00:00 (6810 nps)

Query 32: [p1(X0, X3, a5, X9)]

3 :: 2 - 00:00:00 (2544 nps)

Query 33: [p1(X8, X8, a5, a77)]

3 :: 2 - 00:00:00 (3529 nps)

Query 34: [p6(X9, X9)]

4 :: 3 - 00:00:00 (3641 nps)

Query 35: [p5(a58, a1, X0), p8(X0, X0), p8(a93, X0)]

5 :: 4 - 00:00:00 (4514 nps)

Query 36: [p1(a1, X9, X1, X9)]

3 :: 2 - 00:00:00 (5297 nps)

Query 37: [p1(a78, a0, X0, a19), p1(a12, a0, X0, a53)]

5 :: 4 - 00:00:00 (3228 nps)

Query 38: [p1(a6, a0, X2, X9)]

3 :: 2 - 00:00:00 (2558 nps)

Query 39: [p3(X4)]

2 :: 1 - 00:00:00 (4879 nps)

Query 40: [p8(X3, X3)]

3 :: 2 - 00:00:00 (32922 nps)

Query 41: [p5(a20, a74, X0), p1(X0, a43, a11, a50)]

3 :: 2 - 00:00:00 (3035 nps)

Query 42: [p8(a19, X0), p1(X0, a0, X1, a0), p1(a19, a0, X1, X0), p4(X1, a48)]

7 :: 6 - 00:00:00 (2864 nps)

Query 43: [p1(a71, X3, a5, X2)]

3 :: 2 - 00:00:00 (2765 nps)

Query 44: [p0(X1, X0), p3(X1)]

212 :: 2 - 00:00:00 (1323 nps)

Query 45: [p9(X4, X4)]

2 :: 1 - 00:00:00 (21440 nps)

Query 46: [p6(X5, X0)]

2 :: 1 - 00:00:00 (3824 nps)

Query 47: [p9(X9, X9)]

2 :: 1 - 00:00:00 (8757 nps)

Query 48: [p8(X2, X0), p7(X0, X1), p6(a33, X1), p5(a27, X2, a66)]

1040 :: 4 - 00:00:00 (7743 nps)

Query 49: [p1(a63, X1, a5, X0), p5(a13, a11, X0), p8(a1, X1)]

5 :: 4 - 00:00:00 (4406 nps)

Query 50: [p1(a93, X1, X2, X5)]

3 :: 2 - 00:00:00 (4245 nps)

Query 51: [p9(X6, X6)]

2 :: 1 - 00:00:00 (8745 nps)

Query 52: [p6(a0, X0), p9(X0, a0), p1(X0, a0, a5, a13)]

9 :: 6 - 00:00:00 (5435 nps)

Query 53: [p9(X8, X8)]

2 :: 1 - 00:00:00 (8786 nps)

Query 54: [p0(X1, X0), p9(X0, X0), p5(X2, a0, X1), p5(a1, X3, X2), p4(a17, X3)]

69 :: 6 - 00:00:00 (7354 nps)

Query 55: [p7(X0, X1)]

2 :: 1 - 00:00:00 (4547 nps)

Query 56: [p7(a22, X0), p2(X0, a70, a15)]

3 :: 2 - 00:00:00 (4519 nps)

Query 57: [p9(X9, X9)]

2 :: 1 - 00:00:00 (39843 nps)

Query 58: [p1(a61, a0, X3, X7)]

3 :: 2 - 00:00:00 (4494 nps)

Query 59: [p6(X7, a20)]

2 :: 1 - 00:00:00 (6402 nps)

Query 60: [p0(X0, X1), p7(X0, X2), p8(X1, a19), p2(a19, X3, X2), p2(a57, X3, a15)]

1192 :: 5 - 00:00:00 (9024 nps)

Query 61: [p1(X0, a0, X4, X0)]

4 :: 3 - 00:00:00 (4205 nps)

Query 62: [p1(X3, a0, X5, X3)]

4 :: 3 - 00:00:00 (4355 nps)

Query 63: [p1(X3, a0, X9, X3)]

4 :: 3 - 00:00:00 (11603 nps)

Query 64: [p1(X6, X5, a5, X6)]

4 :: 3 - 00:00:00 (13101 nps)

Query 65: [p6(X0, a17), p2(X1, X0, a35), p7(X0, a0), p0(X1, a0), p8(X0, X0)]

7 :: 6 - 00:00:00 (4399 nps)

Query 66: [p8(X8, X8)]

3 :: 2 - 00:00:00 (4743 nps)

Query 67: [p1(a71, a0, X7, X1)]

3 :: 2 - 00:00:00 (4798 nps)

Query 68: [p2(X0, a1, a49), p1(X0, a0, a5, X0)]

5 :: 4 - 00:00:00 (5233 nps)

Query 69: [p2(X0, a54, a0), p5(X0, a22, a16), p4(a22, X0)]

4 :: 3 - 00:00:00 (5449 nps)

Query 70: [p6(X1, X0), p4(X0, a75), p9(X0, X0), p5(a41, X1, a0)]

27 :: 5 - 00:00:00 (7238 nps)

Query 71: [p0(X0, a4), p5(X0, a11, a50)]

3 :: 2 - 00:00:00 (5510 nps)

Query 72: [p8(X9, X9)]

3 :: 2 - 00:00:00 (4786 nps)

Query 73: [p0(X0, a88), p5(X0, a40, a27)]

3 :: 2 - 00:00:00 (5703 nps)

Query 74: [p1(X0, X1, a5, X0), p2(X1, a14, X0), p1(a77, X1, a5, a10), p9(a91, X1)]

33 :: 7 - 00:00:00 (5025 nps)

Query 75: [p1(a22, X2, a5, a11)]
3 :: 2 - 00:00:00 (6464 nps)

Query 76: [p1(a71, a0, a5, X7)]
3 :: 2 - 00:00:00 (5169 nps)

Query 77: [p1(a0, a0, a5, X6)]
3 :: 2 - 00:00:00 (5150 nps)

Query 78: [p0(X0, X1), p9(X0, X0), p3(X0), p6(X0, a1), p2(X1, a87, a11)]
19040 :: 6 - 00:00:00 (60525 nps)

Query 79: [p1(X0, X1, a5, X0), p2(a44, a20, X0), p8(X1, X1)]
10 :: 6 - 00:00:00 (6008 nps)

Query 80: [p6(a0, X0)]
2 :: 1 - 00:00:00 (40720 nps)

Query 81: [p6(a0, a18)]
4 :: 3 - 00:00:00 (11719 nps)

Query 82: [p6(X0, a16), p2(a48, X0, a34)]
4 :: 2 - 00:00:00 (4844 nps)

Query 83: [p1(X0, X5, X3, X1)]
2 :: 1 - 00:00:00 (4869 nps)

Query 84: [p9(X3, X3)]
2 :: 1 - 00:00:00 (8764 nps)

Query 85: [p8(X5, X5)]
3 :: 2 - 00:00:00 (4749 nps)

Query 86: [p1(X0, X4, X6, X0)]
2 :: 1 - 00:00:00 (5618 nps)

Query 87: [p6(X8, X5)]
2 :: 1 - 00:00:00 (4267 nps)

Query 88: [p1(a58, a0, X2, a11)]
3 :: 2 - 00:00:00 (6317 nps)

Query 89: [p2(X0, a20, a10), p6(X0, a0)]
3 :: 2 - 00:00:00 (4987 nps)

Query 90: [p5(a87, X0, a30), p7(X0, a94)]
3 :: 2 - 00:00:00 (5595 nps)

Query 91: [p2(a62, a29, X0), p6(X0, a19)]
3 :: 2 - 00:00:00 (4499 nps)

Query 92: [p9(X8, X8)]
2 :: 1 - 00:00:00 (42819 nps)

Query 93: [p6(X0, X1), p9(X0, X0), p5(a90, a40, X0), p6(X1, X2), p2(X2, a70, a89)]

98 :: 6 - 00:00:00 (11284 nps)

Query 94: [p1(X8, a0, X4, a13)]
3 :: 2 - 00:00:00 (5876 nps)

Query 95: [p1(X0, a0, a5, X0), p8(X0, a45)]
7 :: 4 - 00:00:00 (7306 nps)

Query 96: [p1(a93, a0, X0, X3)]
3 :: 2 - 00:00:00 (4752 nps)

Query 97: [p8(X7, X7)]
3 :: 2 - 00:00:00 (4743 nps)

Query 98: [p9(X2, X2)]
2 :: 1 - 00:00:00 (8664 nps)

Query 99: [p1(a17, X8, X2, a17)]
4 :: 3 - 00:00:00 (16260 nps)

Query 100: [p7(X0, a86), p5(X0, a22, a16)]
3 :: 2 - 00:00:00 (9839 nps)

std: 222.71 nodes/query

Time to run all queries: 0.8731011380000009

using cuda device

UNITY: ming

Embedding Model: rKB_model.pth

Scoring Model: mod_uni_model.pt

Control: mingoal

Rule Eval: Max Rule

Learned Embedding: Network using cuda

/opt/tljh/user/envs/cse327python311/lib/python3.11/site-packages/torch/nn/modules/lazy.py:180: UserWarning: Lazy modules are a new feature under heavy development so changes to the API or functionality can happen at any moment.

warnings.warn('Lazy modules are a new feature under heavy development '

Query 1: [p2(a12, a72, X0), p0(X0, X1), p3(X1), p8(a63, X1)]
20 :: 15 - 00:00:01 (13 nps)

Query 2: [p1(X1, a0, a5, X0)]
3 :: 2 - 00:00:00 (14 nps)

Query 3: [p0(X1, X0), p6(a37, X0), p8(a77, X1)]
5 :: 3 - 00:00:00 (16 nps)

Query 4: [p1(X7, a0, a5, X7)]
4 :: 3 - 00:00:00 (21 nps)

Query 5: [p1(X1, a0, a5, X1)]
4 :: 3 - 00:00:00 (21 nps)

Query 6: [p1(X2, a0, a5, X6)]
3 :: 2 - 00:00:00 (13 nps)

Query 7: [p6(X0, a18)]

4 :: 3 - 00:00:00 (117 nps)

Query 8: [p2(X0, X1, a92), p6(a94, X0), p5(a0, X1, a39)]
4 :: 3 - 00:00:00 (64 nps)

Query 9: [p8(X3, X3)]
3 :: 2 - 00:00:00 (17 nps)

Query 10: [p5(a20, X0, a24), p1(a0, X0, a14, a10)]
3 :: 2 - 00:00:00 (127 nps)

Query 11: [p1(X6, X1, X7, X6)]
2 :: 1 - 00:00:00 (101 nps)

Query 12: [p9(X5, X5)]
2 :: 1 - 00:00:00 (151 nps)

Query 13: [p8(X1, X1)]
3 :: 2 - 00:00:00 (17 nps)

Query 14: [p4(X0, X1), p8(X0, X0), p2(a14, X1, X2), p8(X2, X2), p0(a0, X2)]
8 :: 7 - 00:00:00 (15 nps)

Query 15: [p8(X0, X0), p1(X0, a0, a5, X0)]
4 :: 3 - 00:00:00 (25 nps)

Query 16: [p9(a27, a27)]
3 :: 2 - 00:00:00 (214 nps)

Query 17: [p3(X0), p1(a41, a58, a10, X0), p9(X0, X0), p2(a75, a54, X0)]
99 :: 15 - 00:00:01 (69 nps)

Query 18: [p1(a1, X6, a5, X6)]
3 :: 2 - 00:00:00 (95 nps)

Query 19: [p1(a17, X1, X3, a17)]
4 :: 3 - 00:00:00 (98 nps)

Query 20: [p5(X0, a10, X1), p2(a40, X0, a11), p9(X1, X1)]
5 :: 4 - 00:00:00 (73 nps)

Query 21: [p1(X6, a0, X5, X6)]
4 :: 3 - 00:00:00 (21 nps)

Query 22: [p3(X7)]
2 :: 1 - 00:00:00 (17 nps)

Query 23: [p2(a98, X1, X0), p4(X0, a70), p7(X1, a35)]
4 :: 3 - 00:00:00 (33 nps)

Query 24: [p9(a27, a27)]
3 :: 2 - 00:00:00 (32735 nps)

Query 25: [p1(a11, a0, a5, a31)]
3 :: 2 - 00:00:00 (95 nps)

Query 26: [p8(a0, X0), p8(X0, X0)]

434 :: 4 - 00:00:04 (93 nps)

Query 27: [p9(a6, a6)]

3 :: 2 - 00:00:00 (213 nps)

Query 28: [p0(X0, a0), p2(X0, X2, X1), p7(a79, X1), p8(X2, X2)]

6 :: 5 - 00:00:00 (6 nps)

Query 29: [p9(X4, X4)]

2 :: 1 - 00:00:00 (137 nps)

Query 30: [p9(a38, X0), p1(a91, X0, a5, a91), p1(a25, X0, a23, a54)]

6 :: 5 - 00:00:00 (44 nps)

Query 31: [p1(X1, a0, X0, X1)]

4 :: 3 - 00:00:00 (308 nps)

Query 32: [p1(X0, X3, a5, X9)]

3 :: 2 - 00:00:00 (13 nps)

Query 33: [p1(X8, X8, a5, a77)]

3 :: 2 - 00:00:00 (90 nps)

Query 34: [p6(X9, X9)]

4 :: 3 - 00:00:00 (133 nps)

Query 35: [p5(a58, a1, X0), p8(X0, X0), p8(a93, X0)]

5 :: 4 - 00:00:00 (36 nps)

Query 36: [p1(a1, X9, X1, X9)]

3 :: 2 - 00:00:00 (242 nps)

Query 37: [p1(a78, a0, X0, a19), p1(a12, a0, X0, a53)]

5 :: 4 - 00:00:00 (41 nps)

Query 38: [p1(a6, a0, X2, X9)]

3 :: 2 - 00:00:00 (75 nps)

Query 39: [p3(X4)]

2 :: 1 - 00:00:00 (16 nps)

Query 40: [p8(X3, X3)]

3 :: 2 - 00:00:00 (25701 nps)

Query 41: [p5(a20, a74, X0), p1(X0, a43, a11, a50)]

3 :: 2 - 00:00:00 (106 nps)

Query 42: [p8(a19, X0), p1(X0, a0, X1, a0), p1(a19, a0, X1, X0), p4(X1, a48)]

48 :: 5 - 00:00:00 (85 nps)

Query 43: [p1(a71, X3, a5, X2)]

3 :: 2 - 00:00:00 (58 nps)

Query 44: [p0(X1, X0), p3(X1)]

3 :: 2 - 00:00:00 (476 nps)

Query 45: [p9(X4, X4)]
2 :: 1 - 00:00:00 (26577 nps)

Query 46: [p6(X5, X0)]
2 :: 1 - 00:00:00 (8 nps)

Query 47: [p9(X9, X9)]
2 :: 1 - 00:00:00 (146 nps)

Query 48: [p8(X2, X0), p7(X0, X1), p6(a33, X1), p5(a27, X2, a66)]
5 :: 4 - 00:00:00 (6 nps)

Query 49: [p1(a63, X1, a5, X0), p5(a13, a11, X0), p8(a1, X1)]
5 :: 4 - 00:00:00 (33 nps)

Query 50: [p1(a93, X1, X2, X5)]
3 :: 2 - 00:00:00 (63 nps)

Query 51: [p9(X6, X6)]
2 :: 1 - 00:00:00 (144 nps)

Query 52: [p6(a0, X0), p9(X0, a0), p1(X0, a0, a5, a13)]
64 :: 5 - 00:00:00 (85 nps)

Query 53: [p9(X8, X8)]
2 :: 1 - 00:00:00 (135 nps)

Query 54: [p0(X1, X0), p9(X0, X0), p5(X2, a0, X1), p5(a1, X3, X2), p4(a17, X3)]
41 :: 6 - 00:00:00 (123 nps)

Query 55: [p7(X0, X1)]
2 :: 1 - 00:00:00 (20709 nps)

Query 56: [p7(a22, X0), p2(X0, a70, a15)]
3 :: 2 - 00:00:00 (45 nps)

Query 57: [p9(X9, X9)]
2 :: 1 - 00:00:00 (26318 nps)

Query 58: [p1(a61, a0, X3, X7)]
3 :: 2 - 00:00:00 (92 nps)

Query 59: [p6(X7, a20)]
2 :: 1 - 00:00:00 (159 nps)

Query 60: [p0(X0, X1), p7(X0, X2), p8(X1, a19), p2(a19, X3, X2), p2(a57, X3, a15)]
6 :: 5 - 00:00:00 (7 nps)

Query 61: [p1(X0, a0, X4, X0)]
4 :: 3 - 00:00:00 (355 nps)

Query 62: [p1(X3, a0, X5, X3)]
4 :: 3 - 00:00:00 (22 nps)

Query 63: [p1(X3, a0, X9, X3)]

4 :: 3 - 00:00:00 (420 nps)

Query 64: [p1(X6, X5, a5, X6)]

4 :: 3 - 00:00:00 (437 nps)

Query 65: [p6(X0, a17), p2(X1, X0, a35), p7(X0, a0), p0(X1, a0), p8(X0, X0)]

7 :: 6 - 00:00:00 (30 nps)

Query 66: [p8(X8, X8)]

3 :: 2 - 00:00:00 (17 nps)

Query 67: [p1(a71, a0, X7, X1)]

3 :: 2 - 00:00:00 (58 nps)

Query 68: [p2(X0, a1, a49), p1(X0, a0, a5, X0)]

5 :: 4 - 00:00:00 (145 nps)

Query 69: [p2(X0, a54, a0), p5(X0, a22, a16), p4(a22, X0)]

4 :: 3 - 00:00:00 (95 nps)

Query 70: [p6(X1, X0), p4(X0, a75), p9(X0, X0), p5(a41, X1, a0)]

10 :: 5 - 00:00:00 (25 nps)

Query 71: [p0(X0, a4), p5(X0, a11, a50)]

3 :: 2 - 00:00:00 (164 nps)

Query 72: [p8(X9, X9)]

3 :: 2 - 00:00:00 (18 nps)

Query 73: [p0(X0, a88), p5(X0, a40, a27)]

3 :: 2 - 00:00:00 (190 nps)

Query 74: [p1(X0, X1, a5, X0), p2(X1, a14, X0), p1(a77, X1, a5, a10), p9(a91, X1)]

8 :: 7 - 00:00:00 (39 nps)

Query 75: [p1(a22, X2, a5, a11)]

3 :: 2 - 00:00:00 (92 nps)

Query 76: [p1(a71, a0, a5, X7)]

3 :: 2 - 00:00:00 (56 nps)

Query 77: [p1(a0, a0, a5, X6)]

3 :: 2 - 00:00:00 (47 nps)

Query 78: [p0(X0, X1), p9(X0, X0), p3(X0), p6(X0, a1), p2(X1, a87, a11)]

280 :: 15 - 00:00:02 (106 nps)

Query 79: [p1(X0, X1, a5, X0), p2(a44, a20, X0), p8(X1, X1)]

7 :: 6 - 00:00:00 (71 nps)

Query 80: [p6(a0, X0)]

2 :: 1 - 00:00:00 (25207 nps)

Query 81: [p6(a0, a18)]

4 :: 3 - 00:00:00 (177 nps)

Query 82: [p6(X0, a16), p2(a48, X0, a34)]

3 :: 2 - 00:00:00 (65 nps)

Query 83: [p1(X0, X5, X3, X1)]

2 :: 1 - 00:00:00 (13 nps)

Query 84: [p9(X3, X3)]

2 :: 1 - 00:00:00 (175 nps)

Query 85: [p8(X5, X5)]

3 :: 2 - 00:00:00 (21 nps)

Query 86: [p1(X0, X4, X6, X0)]

2 :: 1 - 00:00:00 (112 nps)

Query 87: [p6(X8, X5)]

2 :: 1 - 00:00:00 (10 nps)

Query 88: [p1(a58, a0, X2, a11)]

3 :: 2 - 00:00:00 (116 nps)

Query 89: [p2(X0, a20, a10), p6(X0, a0)]

3 :: 2 - 00:00:00 (56 nps)

Query 90: [p5(a87, X0, a30), p7(X0, a94)]

3 :: 2 - 00:00:00 (80 nps)

Query 91: [p2(a62, a29, X0), p6(X0, a19)]

3 :: 2 - 00:00:00 (74 nps)

Query 92: [p9(X8, X8)]

2 :: 1 - 00:00:00 (24473 nps)

Query 93: [p6(X0, X1), p9(X0, X0), p5(a90, a40, X0), p6(X1, X2), p2(X2, a70, a89)]

7 :: 6 - 00:00:00 (10 nps)

Query 94: [p1(X8, a0, X4, a13)]

3 :: 2 - 00:00:00 (94 nps)

Query 95: [p1(X0, a0, a5, X0), p8(X0, a45)]

428 :: 4 - 00:00:00 (487 nps)

Query 96: [p1(a93, a0, X0, X3)]

3 :: 2 - 00:00:00 (82 nps)

Query 97: [p8(X7, X7)]

3 :: 2 - 00:00:00 (21 nps)

Query 98: [p9(X2, X2)]

2 :: 1 - 00:00:00 (175 nps)

Query 99: [p1(a17, X8, X2, a17)]

4 :: 3 - 00:00:00 (418 nps)

Query 100: [p7(X0, a86), p5(X0, a22, a16)]

3 :: 2 - 00:00:00 (63 nps)

ming: 17.39 nodes/query
Time to run all queries: 23.264747566000004

In []: